ABSTRACT

This project investigates the use of a linear regression model, specifically Logistic Regression, on a small and complex biological dataset: the PIMA Indians Diabetes Dataset. The performance of the model was compared with a Random Forest model and an XGBoost model. The dataset was loaded with Kaggle and preprocessed with clinically based decisions. The performance of the models before and after fine-tuning with GridSearchCV was recorded. Due to the imbalance present in the dataset, the F1-score was an important indicator of performance, along with ROC-AUC scores. The fine-tuned Logistic Regression model achieved an F1-score of 0.72, while XGBoost and Random Forest achieved scores of 0.70 and 0.69, respectively. Additionally, the medical nature of the dataset demanded that Recall scores are prioritised. A fine-tuned Logistic Regression model achieved a recall of 87%, while both XGBoost and Random Forest achieved a recall of 81%. These results may be attributed to the robust feature engineering performed on the dataset, indicating that thorough preprocessing enables simpler models to perform effectively.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1 INTRODUCTION & PROBLEM CONTEXT

1.1 Background Information

According to the World Health Organisation, the number of diabetic individuals has risen from 200 million in 1990 to 830 million in 2022, with prevalence increasing at a faster rate in low- and middle-income countries. In 2022, diabetes affected 14% of adults aged 18 years and older. Additionally, it is also found that 59% of diabetic adults aged 30 years and older forsake their medication in 2022. Moreover, nearly half of all diabetes-related deaths occur before individuals reach 70 years of age. In 2021, diabetes was responsible for 1.6 million deaths. Diabetes is also known to cause other ailments such as kidney or cardiovascular diseases (World Health Organisation, 2024). Currently, according to the International Diabetes Federation, in 2024, there were approximately 589 million adults living with diabetes, and it is projected to rise to 583 million by 2050 (International Diabetes Federation, 2025).

1.2 Problem Statement

The need for a robust yet simple diabetes classification model suitable for edge conditions.

The Pima Indian Diabetes dataset presents a significant challenge as a classification problem due to the inherent complexity present in biological factors. While advanced deep learning architectures offer high predictive power, traditional machine learning models such as Logistic Regression remain valuable due to their simplicity and cost-to-performance ratio. However, the effectiveness of these linear models is often compromised by two primary factors: class overlap and imbalance, and non-linear feature interactions.

When heavy class overlap is present, a simple model may struggle to distinguish the information defining one class from another. This results in a classification model with dangerously high false predictions. Heavy imbalances also affect traditional machine learning models as they are more susceptible to biases; they may be more biased towards the majority group.

Furthermore, feature interactions, especially those involving metabolic features, are often nonlinear and complex. This may hinder simpler machine learning models' ability to draw a linear decision boundary that is precise.

The core problem addressed in this project was to determine whether a Logistic Regression model, after being optimised through rigorous feature engineering and addressing class imbalance, could maintain a competitive performance against more complex, non-linear models. This project could serve as a foundation to bridge the gap between model simplicity and the biological noise present in continuous clinical features.

1.3 Objective

This project aims to:

1. Preprocess, analyse, and perform feature engineering on the PIMA Indian dataset to ensure thorough preparation before training machine learning models.
2. Implement and optimise Logistic Regression for robust performance.
3. Implement other models for comparison to determine the limitations of Logistic Regression.
4. Provide insights into model behaviour, evaluate trade-offs, and identify error patterns.

CHAPTER 2 DATASET & PREPROCESSING

2.1 Downloading Dataset

The dataset was downloaded with the *kagglehub* library. Figure 2.1.1 denotes a successful installation of the dataset.

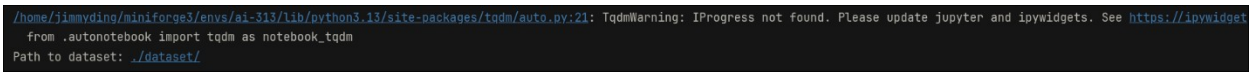


Figure 2.1.1 Successful Installation

2.2 Dataset Description

The immediate follow-up performed in Figure 2.2.1 was to ensure that there were no missing values (null, nan, or empty string). Since the features were primarily crucial medical and clinical records, the presence of *NaN* would have severely disrupted not only the performance and training of the model but also the analysis of the dataset, causing invalid computations.

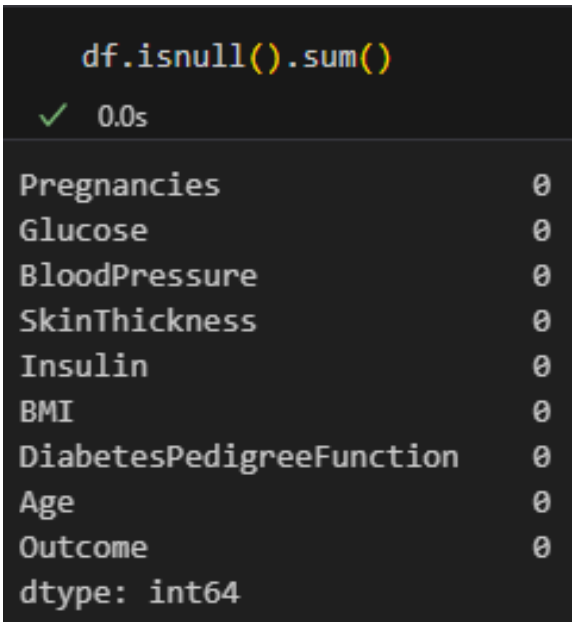


Figure 2.2.1 No NaN Values in The Dataset

Additionally, the *.info()* method in Figure 2.2.2 revealed that the dataset comprised only 768 instances. This relatively small sample size was a critical constraint, as it defined the upper boundary of model complexity. Utilising highly complex models on a dataset of this scale could increase the risk of overfitting, where the model learns the noise of the sample rather than the underlying population trends (IBM, 2021).

```
<class 'pandas.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

Figure 2.2.2 DataFrame Info

Furthermore, the method also revealed that every feature in the dataset was stored numerically (integer and float), indicating that the variables were likely represented as ratio data, where absolute zeros existed in each feature (Bhandari, 2020). This was a crucial detail as it narrowed down the applicable transformation techniques and subsequently allowed us to plan our approach to handling preprocessing early.

However, upon extracting the measurement of central tendencies of the dataset, as shown in Table 2.2.1, the presence of 0 in metabolic indicators such as glucose, blood pressure, skin thickness, and insulin level was indicative of missing values; it is biologically impossible for biological features such as glucose and skin thickness to be 0. This suggested that the missing values in the dataset were stored as 0 or 0.0, as opposed to the conventional *NaN* or *null*.

Table 2.2.1 Measurement of Central Tendencies

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI
count	768.00	768.00	768.00	768.00	768.00	768.00
mean	3.85	120.89	69.11	20.54	79.80	32.00
std	3.37	31.97	19.36	15.95	115.24	7.88
min	0.00	0.00	0.00	0.00	0.00	0.00
25%	1.00	99.00	62.00	0.00	0.00	27.30
50%	3.00	117.00	72.00	23.00	30.50	32.00
75%	6.00	140.25	80.00	32.00	127.25	36.60
max	17.00	199.00	122.00	99.00	846.00	67.10

2.3 Initial Dataset Exploration

Before proceeding with data cleaning, it is essential for an initial data exploration to be conducted, as it provides clues and intuition on the type of techniques and the sequence in which they should be implemented.

The distribution of continuous features heavily affects learning and subsequently the performance of any machine learning model. By determining the direction and magnitude of skewness, we can determine the most robust transformation techniques; simultaneously, it allows us to determine the suitability of certain imputation techniques. According to Figure 2.3.1, one major observable recurring issue was the spikes in frequency count for value zero in features such as glucose, blood pressure, skin thickness, insulin, and BMI. This further solidifies the initial intuition where the missing values were stored as 0 or 0.0 rather than *NaN* or *null*. Therefore, before analysis, imputation, or training can be carried out, these values must be converted to *NaN* first.

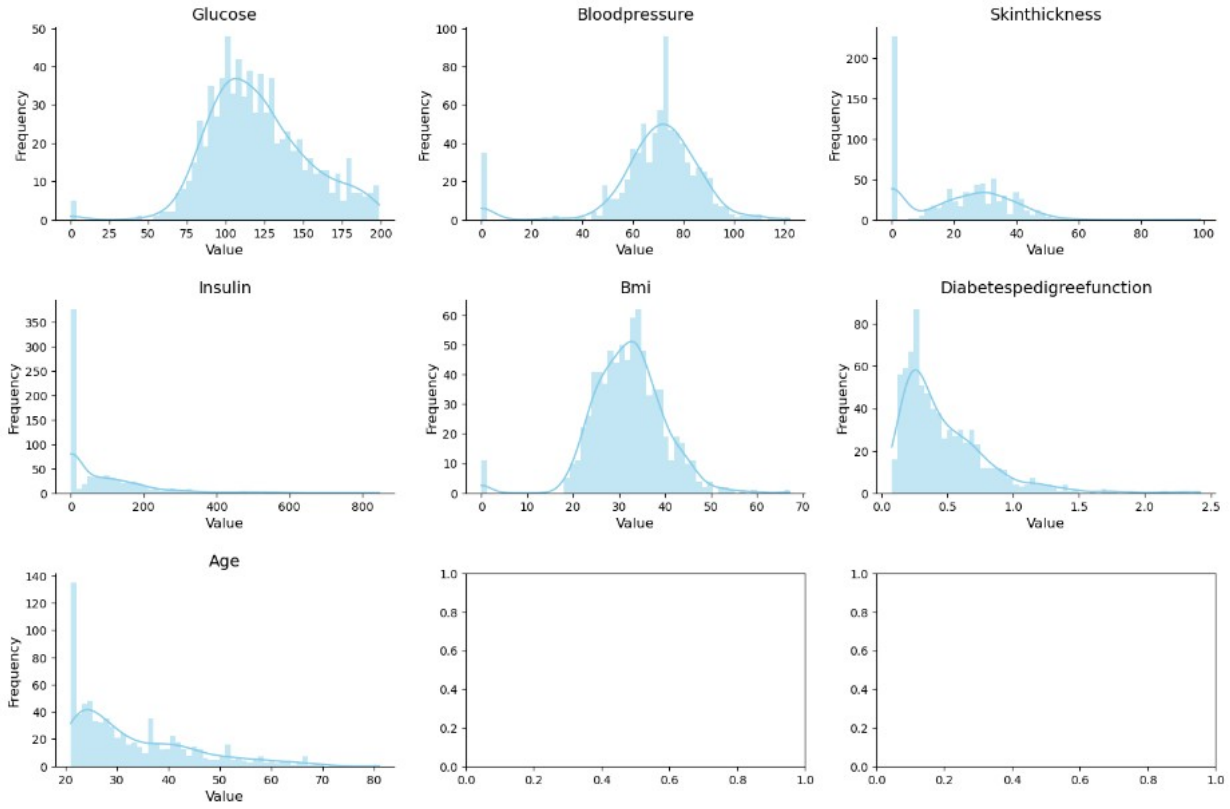


Figure 2.3.1 Distributions of Continuous Features

It is not only the direction and skewness of the features that influence the choice of imputation techniques, but also the percentage of missing values in each feature. Upon searching, it was found that nearly half of the dataset's instances had missing insulin levels, ruling out conventional imputation techniques relying on measures of central tendency. Furthermore, nearly 30% of instances had missing skin thickness. Fortunately, this trend was only limited to only two features, while the remaining features had less than 5% of missing values.

Table 2.3.1 Percentage of Missing Values

Feature	Percentage
Glucose	0.65
Blood Pressure	4.55
Skin Thickness	29.56
Insulin	48.70
BMI	1.43

The distribution of the continuous features remained similar after converting the zeros to *NaN* (Figure 2.3.2); however, the exact skewness of the features became more accurate.

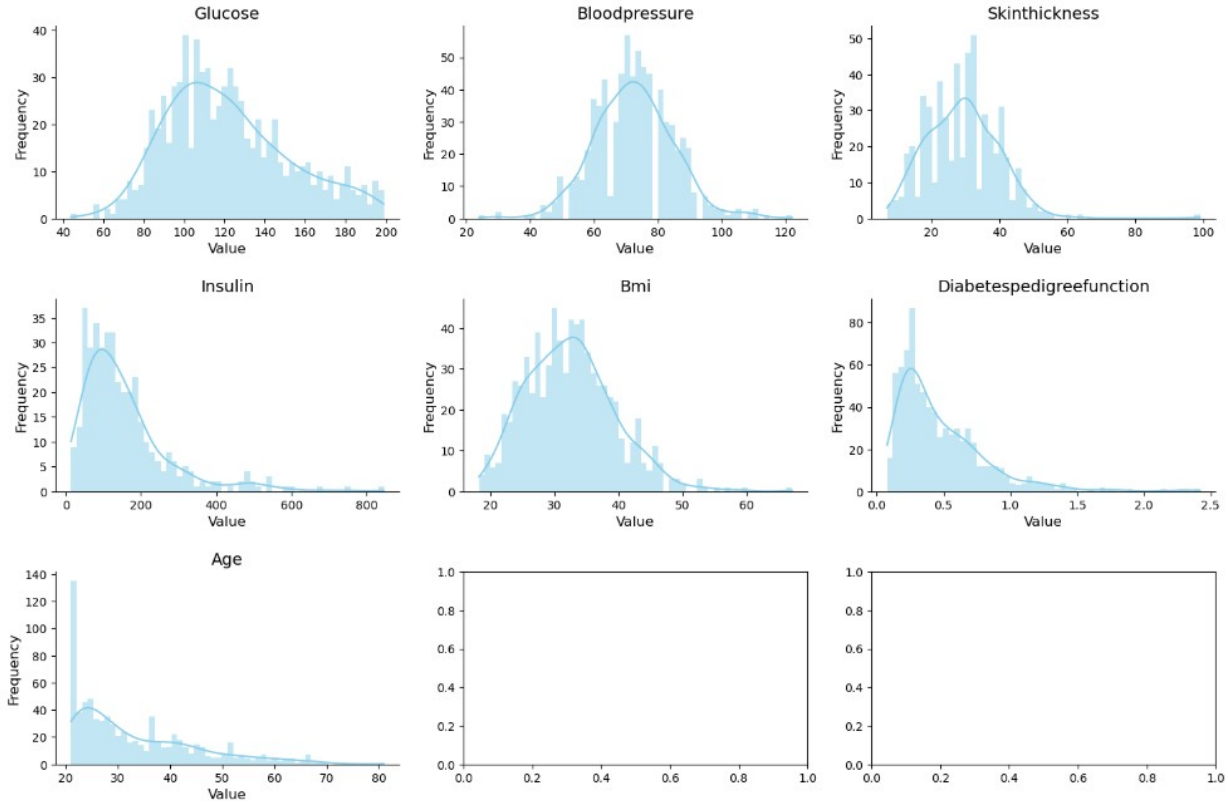


Figure 2.3.2 Distributions of Continuous Features Post Conversion

The distribution of the label was also determined before deciding on splitting techniques; the distribution outcome was imbalanced (65:35 ratio), as shown in Figure 2.3.3.

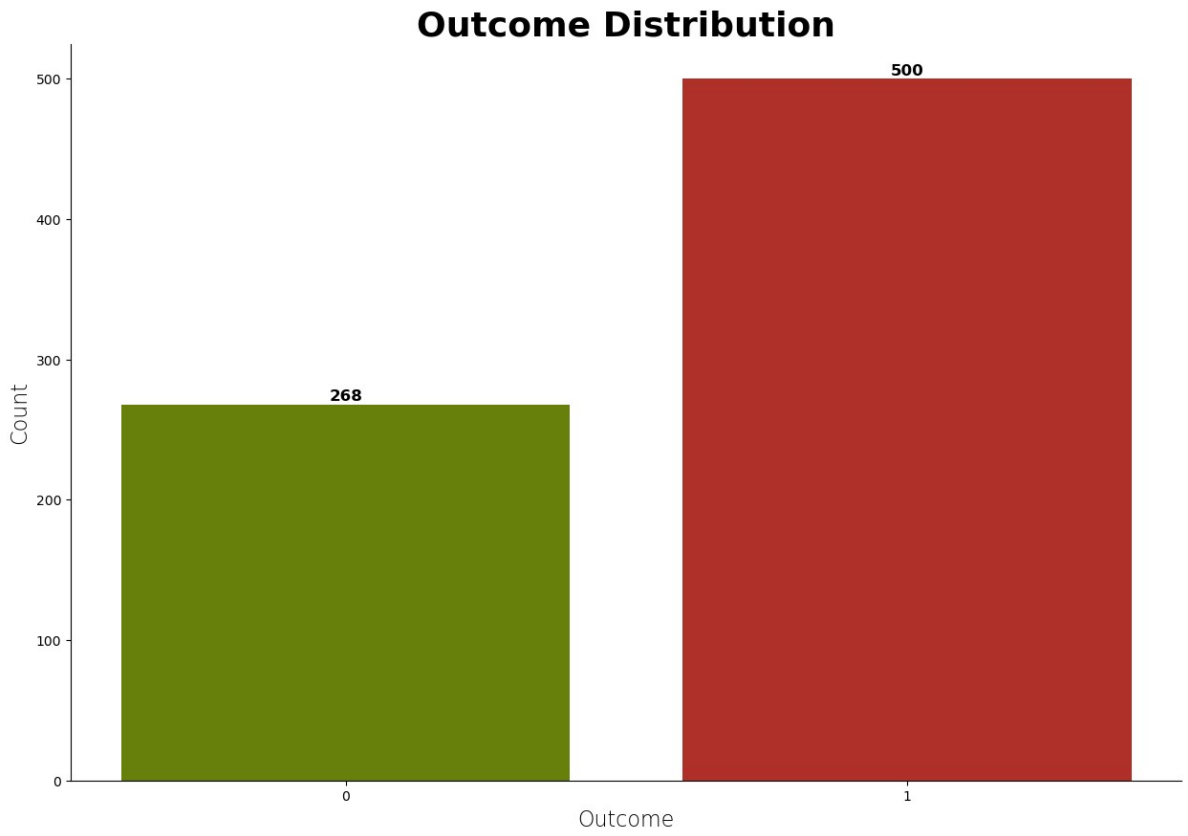


Figure 2.3.3 Outcome Distribution

Although this provided the model with more instances containing patterns correlated to diabetes, this may backfire and cause the model to be more biased towards the majority class. As a result, the false positive cases may rise. Fortunately, this can be mitigated with the use of stratified sampling, where the total population is divided into non-overlapping subgroups based on a specific characteristic before drawing random samples, preserving the original proportions found in the population (Thomas, 2023). While resampling techniques such as SMOTE were considered to address the wide imbalance, they were ultimately rejected due to the risks of overfitting and the creation of artificial feature relationships in a small dataset (Elkan, 2001).

2.4 Dataset Splitting

The dataset was split into a training set (80%) and a test set (20%) with stratified sampling based on the label, providing the model with sufficient, representative data during training (Figure 2.4.1).

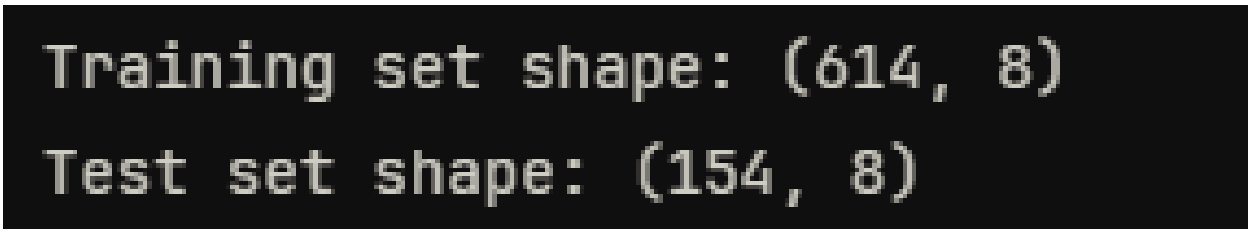


Figure 2.4.1 Train Test Split

2.5 Handling Null Values

To ensure that there were no data leakages, the subsequent preprocessing and feature engineering were conducted on the training and test sets separately. Referring to the prior analysis conducted on the distribution of features and the percentages of missing values, we deduced that *KNNImputer* was the best choice; unlike simple statistical imputation, which treats each feature in isolation, KNN Imputation leverages the local similarity between instances to estimate missing values. For instance, a missing insulin value for a high-BMI individual is more accurately estimated using other high-BMI individuals rather than the global median of the entire population.

The algorithm identifies the *k* most similar instances to the row with missing data using a distance metric. The missing values are then replaced by the mean value of that specific feature from the identified neighbours (Scikit-Learn, 2025). For this relatively small dataset, a value of k = 5 was selected to maintain a balance between local accuracy and the reduction of noise in a small sample.

To ensure that every feature contributed equally to the KNN Imputation, z-score standardisation was applied to the dataset. If standardisation were not applied, features with larger values, such as glucose level would dominate the distance calculation, resulting in a mathematically biased neighbour search. This transformation rescaled the features, so they had a mean of 0 and standard deviation of 1, according to the formula:

$$z = \frac{x - \mu}{\sigma}$$

2.6 Feature Engineering

2.6.1 Derived Additional Features

Before any feature engineering could be carried out, the initial scaling implemented for KNN Imputation had to be reversed. This reversal was necessary to ensure that the new features contain values that were calculated on real-world scales, remaining biologically meaningful. Upon analysis of the dataset, there were seven additional features that we believe will contribute to the model's robustness and interpretability.

Insulin_Glucose_Interaction

This feature provides the model with the level of insulin needed relative to body mass when handling glucose, replacing the conventional insulin-glucose ratio as it scales dynamically with the weight of the individual. This feature can be represented with:

$$Proxy = \frac{Insulin \times Glucose}{BMI}$$

where:

- *Insulin* is insulin levels
- *Glucose* is glucose levels
- *BMI* is Body Mass Index

Glucose_Age_Ratio

This feature normalises glucose burden by age, since older individuals may naturally have different baselines compared to younger individuals. The mathematical representation is:

$$Ratio = \frac{Glucose}{Age}$$

where:

- *Glucose* is glucose levels
- *Age* is the age of the individual

Glucose_BMI_Penalty

This is necessary to capture the synergistic penalty of high sugar and high body fat. The following equation shows how it is calculated.

$$Penalty = Glucose \times BMI$$

where:

- *Glucose* is glucose levels
- *BMI* is Body Mass Index

Obesity_IR_Score

Although the thickness of the skin does not directly correlatewith diabetes, it affects insulin resistance, which ultimately influences diabetes risk.

$$Score = BMI \times Skin\ Thickness$$

where:

- *BMI* is Body Mass Index
- *Skin Thickness* is the thickness of the skin measured in ??

Gen_Diabetes_Risk

This feature involves calculating the influence of genetics on an individual's risk of diabetes. It's calculated by weighing family history with the individual's glucose level.

$$Risk = Diabetes\ Pedigree\ Function \times Glucose$$

where:

- *Diabetes Pedigree Function* is a continuous score, ranging from 0.08 to 2.42
- *Glucose* is glucose levels

Adjusted_Pregnant_BMI

Normalises BMI by mathematically accounting for pregnancy as a clinical confounder in long-term body mass. This provides a more realistic interpretation of BMI by factoring in long-term weight retention and changes in body composition as a result of pregnancy.

$$Adjustment = \frac{BMI}{1 + \text{ Pregnancies}}$$

where:

- *BMI* is Body Mass Index
- *Pregnancies* is pregnancy count

Has_Metabolic_Syndrome

To improve the model's sensitivity towards phenotypes, two binary features were engineered. These features represent complex interactions between physiological markers that signal specific signals of metabolic failure.

The first flag is metabolic syndrome cluster, $?_{??}$, where it identifies heavy threats – hyperglycaemia, obesity, and hypertension. While each factor is a risk, their co-occurrence creates synergy. By binarising this cluster of threats, the model can assign a discrete penalty weight, which is more predictive than treating the variables as independent. This binarisation can be represented with,

$$?_{?2}(?, ?, ?) = \begin{cases} 1, ?? (? > 100) \ ? (? > 30) \ ? (? > 80) \\ 0, \quad ??h????? \end{cases}$$

where:

- *G* is glucose level
- *B* is Body Mass Index
- *P* is blood pressure

The second flag, designated as beta-cell exhaustion, identifies instances characterised by high glucose levels in the presence of poor insulin production. In a healthy individual, elevated glucose triggers a similarly high insulin level; therefore, a high glucose level accompanied by a low insulin level is strongly indicative of beta-cell failure. This binary flag explicitly points the model towards individuals whose endocrine system has been exhausted, improving the model's discriminatory power and interpretability.

$$?_{?2}(?, ?) = \begin{cases} 1, ?? (?_{???????} > 140) \ ? (? < 30) \\ 0, \quad ??h????? \end{cases}$$

where:

- *G* is glucose level
- *I* is insulin level

2.6.2 Binning

In addition to continuous transformation, certain clinical variables, such as Body Mass Index (BMI) and Blood Pressure, possess well-established thresholds that outline ranges from normal to critical health states. By discretising these continuous features into bins aligned with medically recognised thresholds, we leverage human-derived thresholds to enhance the model's interpretability and ensure that predictions remain anchored in clinically meaningful scales.

Body Mass Index (BMI) categories defined by the Centers for Disease Control and Prevention (CDC) (Centers for Disease Control and Prevention, 2024) provide clinically recognised cut-offs for adults aged 20 and above. As illustrated in Table 2.6.2.1, the BMI feature was partitioned into six categories: Normal, Pre-Overweight, Overweight, Obesity Class 1, Obesity Class 2, and Severe Obesity. This binarisation was implemented using *pandas.cut* method with labels, with the following bin ranges and labels:

Table 2.6.2.1 Derived BMI Category

BMI Category	**BMI Range (kg/m2)**
Normal	BMI < 20
Pre-Overweight	20 ≤ BMI < 25
Overweight	25 ≤ BMI < 30
Obesity Class 1	30 ≤ BMI < 35
Obesity Class 2	35 ≤ BMI < 40
Severe Obesity (Class 3)	BMI ≥ 40

Similar to BMI, Blood Pressure (BP) is a clinical variable with established thresholds that outline ranges from normal to hypertensive states. According to the National Heart, Lung, and Blood Institute (NHLBI) (National Heart, Lung, and Blood Institute, 2024), blood pressure categories specify clinically recognised thresholds that stratify risk levels for adults aged 20 and above. By categorising the BP feature into 4 categories: Normal, Pre-Hypertension, Hypertension Stage 1

and Hypertension Stage 2. This binarisation was implemented using *pandas.cut* method, with the following bin ranges and labels:

Table 2.6.2.2 Derived BP category for Systolic BP Range

BP Category	**Systolic BP Range (mmHg)**
Normal	BP < 75
Pre-Hypertension	75 ≤ BP < 80
Hypertension Stage 1	80 ≤ BP < 90
Hypertension Stage 2	BP ≥ 90

Both BMI and BP binarisation had included “Pre-” stage to capture borderline cases that fall just below the conventional overweight threshold, thereby improving sensitivity to early risk signs.

Lastly, dividing age into bins as shown in Table 2.6.2.3 allows the model to distinguish between early to young, middle-aged, and older populations. This binarisation was applied using *pandas.cut* method, with the following bin ranges and labels:

Table 2.6.2.3 Age Category

Age Category	**Age Range (years)**
Early Adult	21 – 22
Young Adult	23 – 25
Early Middle Age	26 – 29
Middle Age	30 – 37
Late Middle Age	38 – 53
Senior	> 54

2.6.3 Transformation and Scaling

Based on the initial distribution of continuous features (Figure 2.3.2) and the subsequent visualisation of engineered features (Figure 2.6.3.1), it can be observed that some of the features were heavily right-skewed. To mitigate the risk of high-leverage outliers biasing the model, these features underwent non-linear transformations to approximate a Gaussian distribution. While visual inspection provides an initial heuristic for skewness, a quantitative assessment was

performed with *pandas*'s skew method, which uses Fisher-Pearson Coefficient of Skewness ($?_1$) to determine the precise degree of skewness (pandas-dev, 2025).

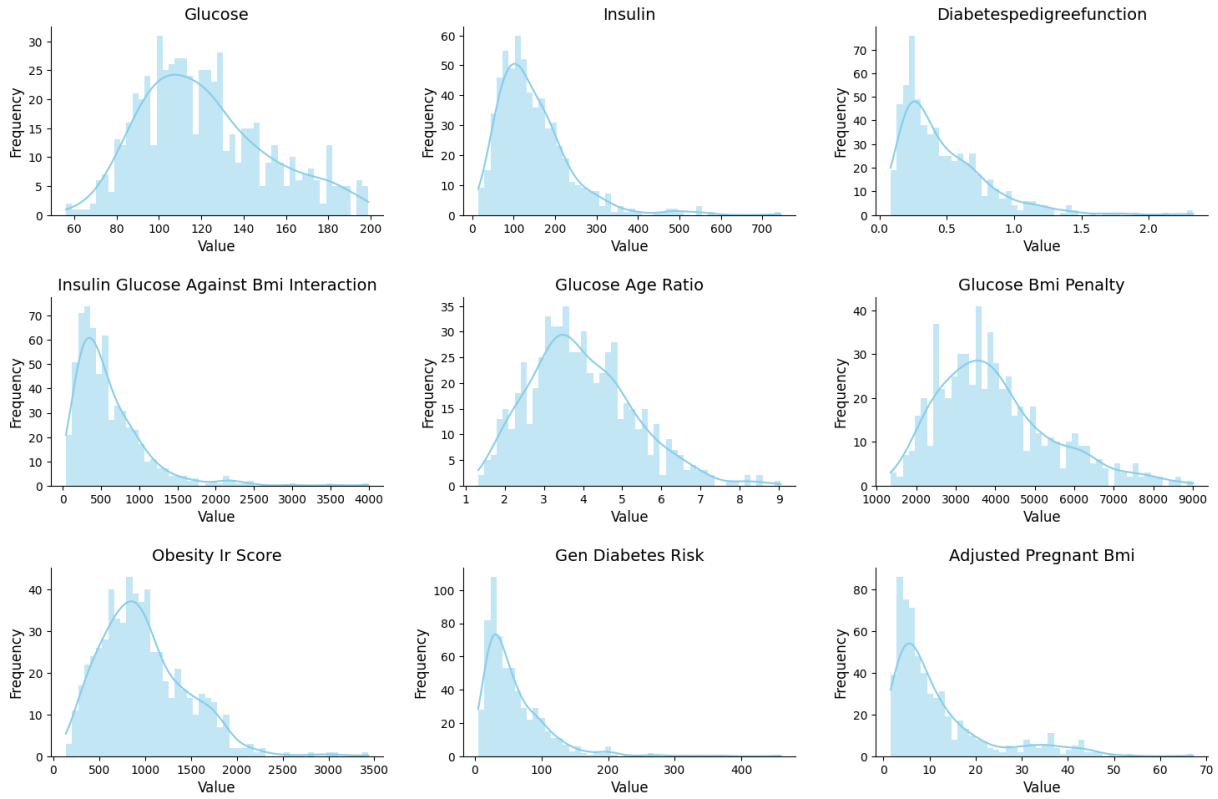


Figure 2.6.3.1 Distributions of Continuous Features Post Feature Engineering

Table 2.6.3.1 shows the exact skewness of each feature.

Table 2.6.3.1 Fisher-Pearson Coefficient of Skewness Result

Feature	Skewness	Transformation Required
Glucose	0.5594	
Insulin	1.9486	
DiabetesPedigreeFunction	1.8189	
Insulin_Glucose_Against_BMI_Interaction	2.3930	
Glucose_Age_Ratio	0.6132	
Glucose_BMI_Penalty	0.8394	

Obesity_IR_Score	0.9814
Gen_Diabetes_Risk	2.7953
Adjustd_Pregnant_BMI	1.7715

An analysis of these results reveals that while glucose and glucose-age ratio exhibited moderate skewness, they fall within an acceptable range. Conversely, highly skewed features underwent a Yeo-Johnson power transformation, selected for its robustness in handling real numbers, including near-zero values present in engineered features (Yeo & Johnson, 2000).

As shown in Table 2.6.3.2 and Figure 2.6.3.2, the transformation successfully reduced skewness across all targeted features to near-zero levels, with *Gen_Diabetes_Risk* improving from 2.7953 to 0.0078.

Table 2.6.3.2 Yeo-Johnson Power Transformation Result

Feature	Skewness
Insulin	0.0101
DiabetesPedigreeFunction	0.1359
Insulin_Glucose_Against_BMI_Interaction	0.0016
Glucose_BMI_Penalty	-0.0011
Obesity_IR_Score	-0.0107
Gen_Diabetes_Risk	0.0078
Adjustd_Pregnant_BMI	0.0780

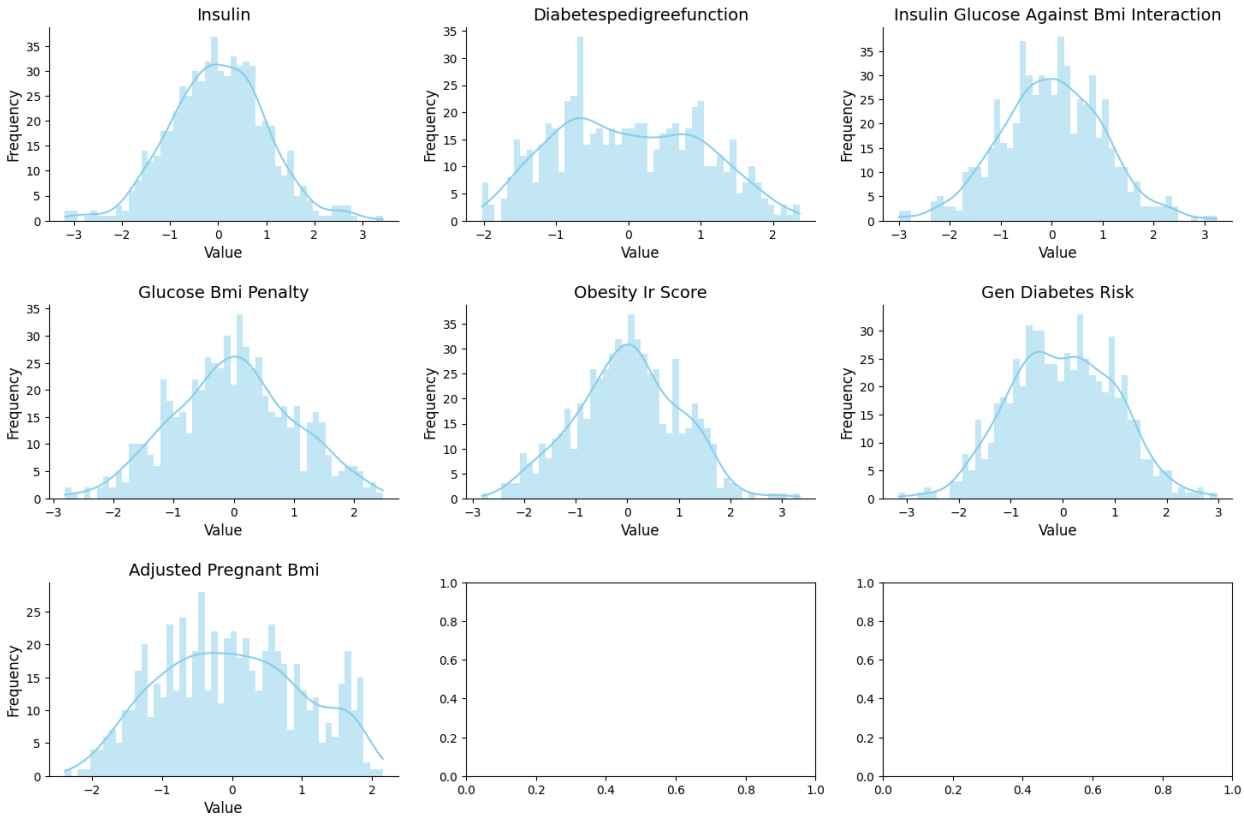


Figure 2.6.3.2 Distribution of Continuous Features Post Transformation

Following the transformation, the dataset underwent z-score standardisation to enforce a uniform unit variance across the feature space. This was necessary to prevent numerical bias towards features with inherently larger variances.

CHAPTER 3 Exploratory Data Analysis

The following analyses were primarily conducted on the dataset prior to preprocessing to ensure the statistical honesty of the dataset was preserved.

3.1 Linearity Between Features

The heatmap of Pearson Correlation, as shown in Figure 3.3.1, demonstrated weak positive linear relationships amongst the features. Consequently, models that rely on linearity are expected to perform poorer than more complex models (eXtreme Gradient Boosting or Random Forest) capable of extracting information from non-linear features.

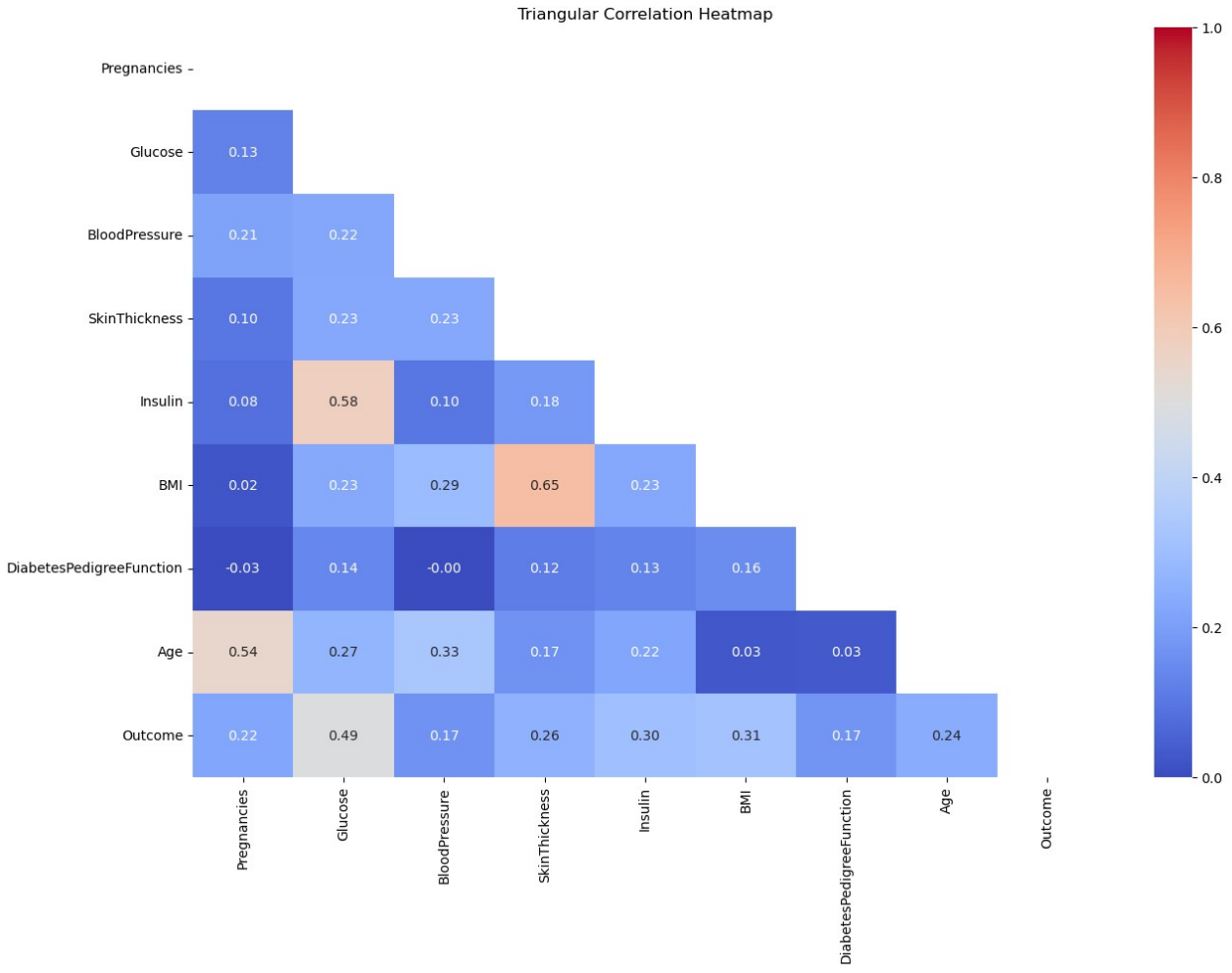


Figure 3.1.1 Heatmap of Pearson Correlation

3.2 Distribution of Continuous Features

The analysis of the distribution of the continuous features in Chapter 2.3 can be expanded by highlighting the effects on the performance of the model.

Based on Figure 2.3.2, although the distribution of glucose is normal, the average level of glucose was still at a dangerous level (between and). According to (Yogish Kudva, 2024), a fasting blood sugar level from 100 to 125 is considered diabetic. Thus, while the non-diabetics were not considered diabetic, they were at high risk of diabetes. The high glucose value for both diabetic and non-diabetic instances results in significant overlaps in data; this may be detrimental to simple models such as Logistic Regression, as it hinders its ability to draw a clear line between the two outcomes. Additionally, the higher average for blood pressure and BMI raises certain concerns as well; there is a massive overlap in values for both diabetic and non-diabetic individuals. The majority of the individuals in the dataset are young adults, with a high concentration in the age group 21, dominating and creating an extreme skewness towards this younger age group, potentially worsening the bias that a model may adopt.

After calculating the skewness of each column, we can determine exactly how skewed they are, allowing us to pick the most robust and appropriate transformation technique during preprocessing. According to Table 3.2.1 and Figure 2.3.2, glucose, blood pressure and BMI are generally normally distributed (values ranging from 0 to 0.5), while skin thickness is skewed by only a small margin. However, *Insulin* is extremely skewed with a score of 2.1665.

Table 3.2.1 Skewness Score

Feature	Skewness
Glucose	0.5310
Blood Pressure	0.1342
Skin Thickness	0.6906
Insulin	2.1665
BMI	0.5940

3.3 Bivariate Analysis

3.3.1 Features Vs Outcome

Based on the violin graphs shown in Figure 3.3.1.1, we can derive some insights:

Glucose Vs Outcome

Individuals who were not diabetic had lower levels of glucose in comparison to those with diabetes; the majority of those who were diabetic had their levels concentrated around 100, whilst those with diabetes had their levels exceeding the upper bound of pre-diabetic, 125 ??/?? .

According to (Yogish Kudva, 2024), a fasting blood sugar level from 100 to 12?/?? is considered pre-diabetic. So, while the non-diabetics have yet to be declared as medically diabetic, they were at high risk of diabetes. Additionally, notice that the distribution of glucose level amongst those who are diabetic is more varied compared to the group who wasn't diabetic; diabetes disrupts the body's natural mechanism of handling blood sugar, causing dangerous fluctuations in glucose level (Suh & Kim, 2015).

The dangerously high glucose levels found in non-diabetics and the larger range of glucose levels within the diabetic group causes a huge overlap in data, further exemplifying the complexity of biological data.

Age vs Outcome

The demographic of those who are not diabetic was young adults, whilst the ages amongst those who were diabetic were much more varied, creating an overlap in data. This makes it difficult for simple models to derive meaningful distinctions.

Diabetes Pedigree function vs Outcome

Diabetes pedigree function (DPF) refers to the genetic component when gauging one's risk of contracting diabetes. Similarly, a big overlap exists between diabetic and non-diabetic groups. Because of this, viewing only DPF against the outcome is ineffective in deriving analyses.

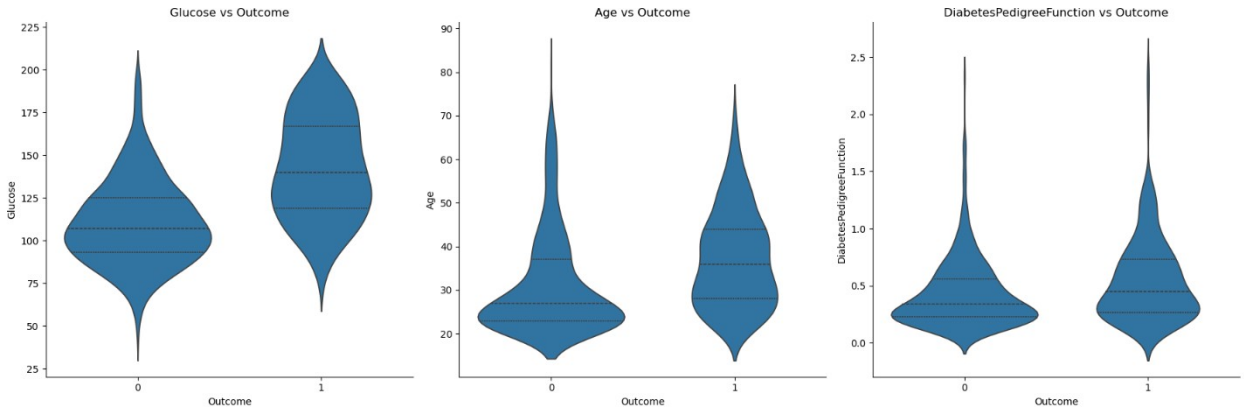


Figure 3.3.1.1 Features Vs Outcome Violin Graphs

3.3.2 Features Vs Features

Visually, the relationship between each pair of features is a weak positive linear relationship (Figure 3.3.2.1). Nonetheless, to be sure, Pearson correlation and Spearman's rank-order correlation are calculated.

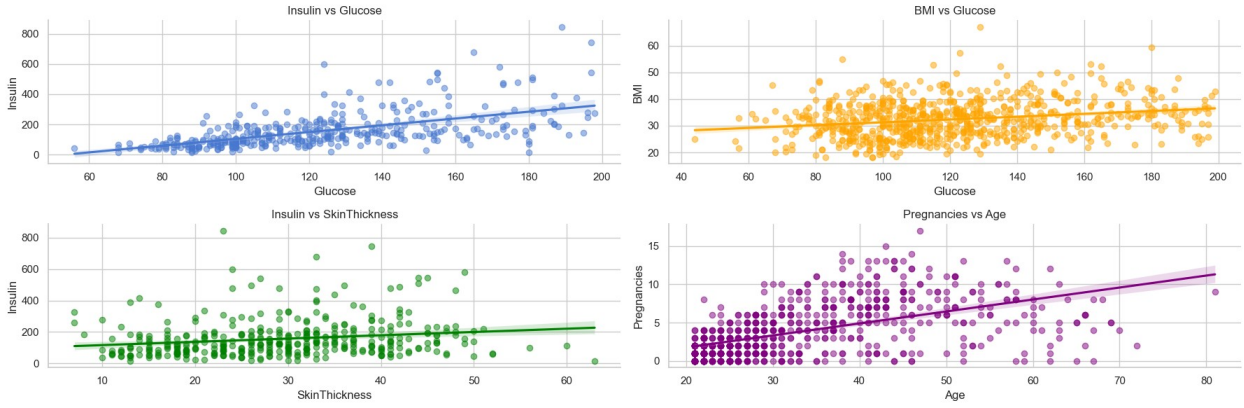


Figure 3.3.2.1 Features Vs Features Regression Graphs

Interpreting Pearson Correlation alone provides us with the exact strength of the positive relationships. According to Table 3.3.2.1, most of these pairs have weak to moderate linear relationships with one another.

Table 3.3.2.1 Pearson Correlation Scores

Feature	Pearson Correlation
Glucose and BMI	0.233
Glucose and Insulin	0.581
Age and Pregnancies	0.544
Insulin and Skin Thickness	0.185

This weak-to-moderate linear relationship trend is supported by the low Spearman rank-order correlation. The low values, as shown in Table 3.2.2.2, in certain features prove that the relationships between certain pairs may not be directly proportional, solidifying the conjecture of high data complexity.

Table 3.3.2.2 Spearman Correlation Score

Feature	Spearman’s Correlation
Glucose and BMI	0.228
Glucose and Insulin	0.659
Age and Pregnancies	0.607
Insulin and Skin Thickness	0.245

The fact that Spearman correlation is consistently higher than Pearson's indicates that monotonic relationships exist, even if the linear strength in each pair is only low to moderate. Despite having low scores in individual computations, the difference between them, albeit small, still provides us with an indication of the type of relationship.

Table 3.3.2.3 Gap Between Pearson Correlation and Spearman Correlation

Feature	Difference
Glucose and BMI	0.005
Glucose and Insulin	0.078
Age and Pregnancies	0.063
Insulin and Skin Thickness	0.060

The marginal non-linearity gap across all pairs suggests that while the relationships are inherently complex and noisy, they follow a monotonic trend but not strictly linear. The higher Spearman values indicate that the variables tend to move in the same direction, but the rate of change is inconsistent.

3.4 Multivariate Analysis

Each of the scatter plots involves outcome as the hue of the plot points, allowing us to view the distributions, relationships, and clusters that can be found when comparing different features simultaneously.

Based on the scatter plot (Figure 3.4.1) depicting glucose, BMI, and outcome, there exists a separation between the diabetics and the non-diabetics, albeit a big overlap between these two groups can be observed too, and the plot points are sparse. This is detrimental to simpler models, as this overlap and sparseness make it difficult to achieve high accuracy and precision. The sparseness also suggests that while these factors do increase the risk of diabetes, they are not indicative. Nonetheless, the scatter denotes a pattern - those who are not diabetic have lower glucose levels and lower BMI compared to those who are diabetic.

In the scatter plot involving glucose, insulin, and outcome, we can observe a linear pattern, which is statistically supported by a Pearson correlation of 0.581. When factored by Outcome, a clear gradient is visible; non-diabetic instances are concentrated at the lower-left origin, while diabetic instances shift toward the upper-right. This linear alignment suggests that while the two features are co-dependent, the significant overlap in the mid-range indicates that a simple linear threshold may be insufficient for perfect classification, necessitating a more thorough feature engineering to simplify the complexities of the dataset.

The vertical pattern found in the scatter plot illustrating age, pregnancies, and outcome proves that these three variables have a minimal relationship.

The pattern found in the final scatter plot (glucose vs diabetes pedigree function and outcome) is similar to the pattern found when glucose and BMI were involved instead. This suggests that while these two variables are not a guarantee of diabetes, they still increase the risk of contracting diabetes.

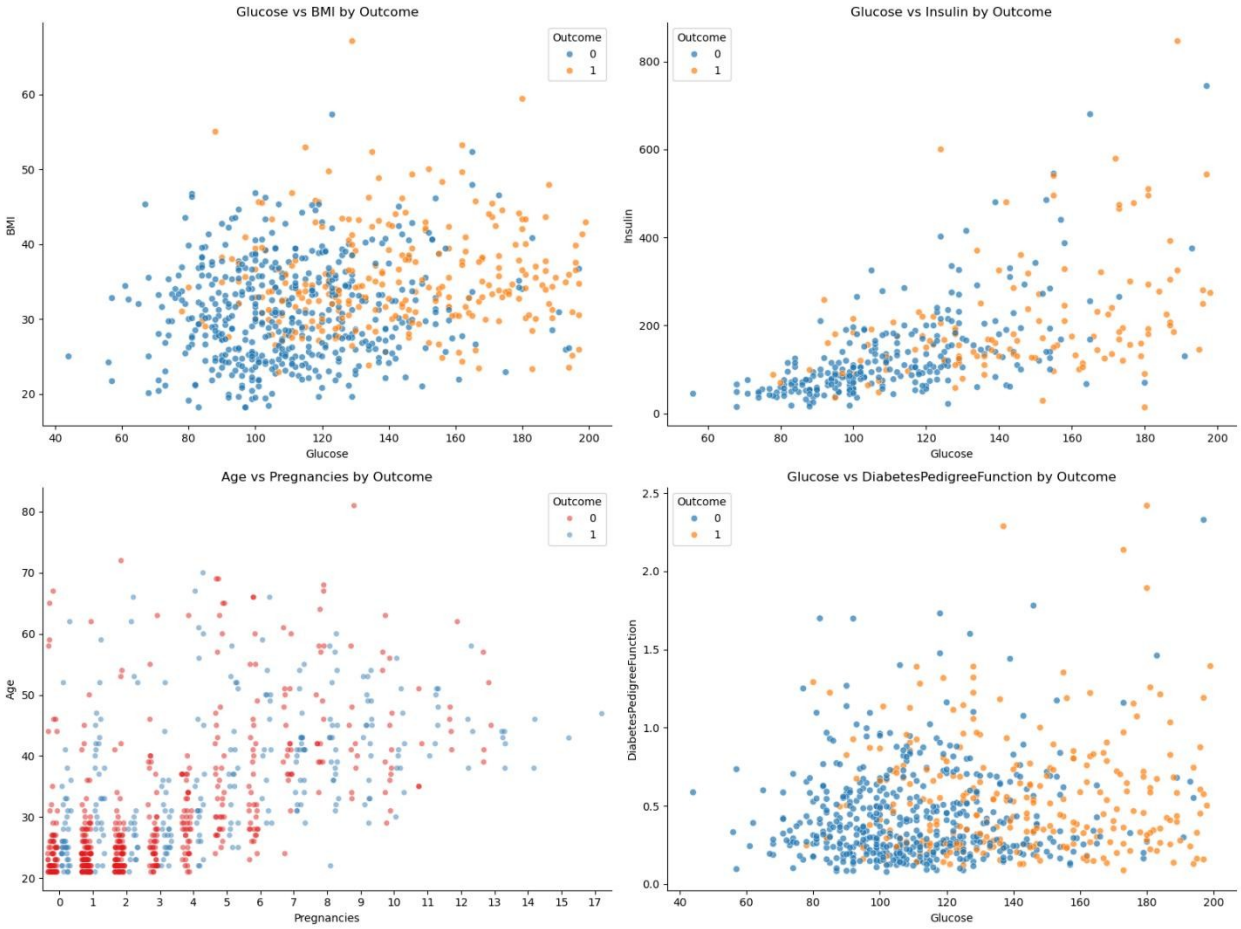


Figure 3.4.1 Multivariate Features Vs Outcome Scatter Plot A

Additional scatter plots were visualised to extract more interesting context from the dataset (Figure 3.4.2), although the extracted insights may not have a direct impact on the performance of the model.

The insulin vs glucose distribution reveals a dense central cluster where skin thickness remains consistently between 20 and 40□□ . The sparse instances of extreme skin thickness appearing as outliers outside the main concentration likely contribute to the lower Pearson correlation. This suggests that while skin thickness is biologically related to the other metabolic indicators (insulin and glucose), its high variance may limit its individual predictive power.

The vertical distribution in the pregnancies vs glucose plot indicates a high degree of independence, as glucose levels do not show a significant linear shift as pregnancy count increases. However, a clinically significant cluster is observed: most subjects maintain a pre-diabetic glucose baseline of

100-120 □□/□□ regardless of pregnancy history. Furthermore, the low variance in blood pressure (60 - 80 □□□□) across the entire dataset suggests this feature may offer limited discriminative information for the model's classification task.

Finally, a strong multivariable trend is identifiable in the insulin vs glucose plot when BMI is factored in. A clear positive monotonic progression is visible; as glucose and insulin levels rise, there is a corresponding increase in BMI. With a high population-wide average BMI of 30, this cluster represents the primary risk profile within the dataset.

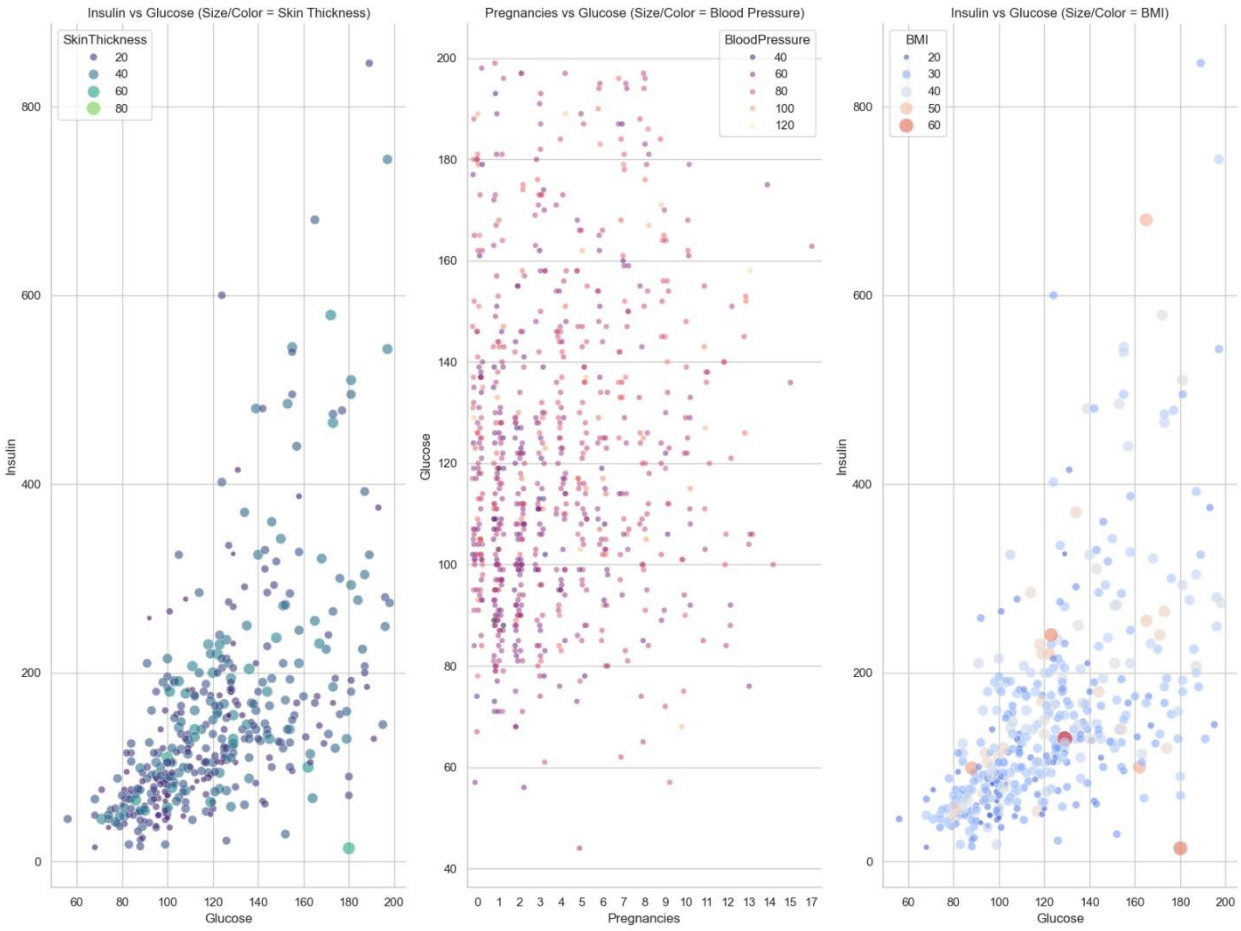


Figure 3.4.2 Multivariate Features Vs Outcome Scatter Plot B

3.5 Conclusion

In conclusion, it is found that the relationships between features in this dataset are predominantly non-linear and overlapping. This is a critical observation, as models that rely on the assumption of linearity, such as Logistic Regression, will struggle to define a clear decision boundary, unless a thorough and robust feature engineering is carried out.

This mathematical difficulty reflects the underlying clinical profile of the Pima Indian sample. This dataset contains a significant class imbalance, and the combination of high average BMI and elevated glucose levels alongside relatively low insulin levels serves as a strong indicator of insulin resistance. Because these factors exist on a continuum rather than in distinct groups, the resulting data clusters overlap, making it difficult for simple models to distinguish between diabetic and non-diabetic individuals correctly.

CHAPTER 4 MODEL DEVELOPMENT

4.1 Primary Model, Techniques and Evaluation (Logistic Regression)

The Pima Indian Diabetes dataset was a dataset with two target outcomes: diabetic negative (0) and diabetic positive (1), a binary classification problem. Therefore, a binary classification model such as Logistic Regression was selected as our main model to predict diabetic status, and its performance will be evaluated against the two alternative models in Section 4.2.

Helper Function

Before the training process, a helper function *plotcm_classification_report()* in Figure 4.1.1 was defined to bundle Scikit-learn's Confusion Matrix and Classification Report to streamline the notebook and provide a consistent way to evaluate the model performance.

```
def plotcm_classification_report(y_pred, y_test):
    cm = confusion_matrix(y_test, y_pred)

    plt.figure(figsize=(6, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=['Predicted Healthy', 'Predicted Diabetic'],
                yticklabels=['Actual Healthy', 'Actual Diabetic'])
    plt.title('Diabetes Prediction Results')
    plt.show()

    print(classification_report(y_test, y_pred))
```

Figure 4.1.1 Function Bundled with Confusion Matrix and Classification Report

Technique: Train baseline model and evaluate performance

To train this primary model, the preprocessed data splits (X_train, X_test, y_train, and y_test) were loaded to train a baseline Logistic Regression classifier as shown in Figure 4.1.2. During model initialisation, to address the identified class imbalance within the dataset, the *class_weight* parameter was set to "balanced". In addition, the *max_iter* parameter value was set to 1000 to ensure the optimisation algorithm successfully converges during training. The model was then trained by fitting the training set (X_train and y_train) to the Logistic Regression classifier.

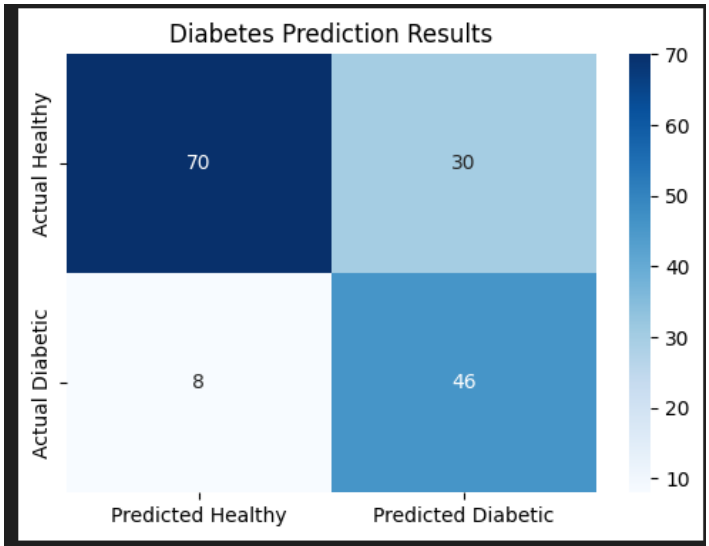
```
model = LogisticRegression(class_weight="balanced", max_iter=1000)
model.fit(X_train, y_train)
```

Figure 4.1.2 Training Baseline Logistic Regression

After training, the model trained on the training datasets was used to predict on the test dataset (X_test and y_test), and the predictions were evaluated using *plotcm_classification_report()* as shown in Figure 4.1.3. After execution, a performance report consisting of a confusion matrix and a classification report was produced for the base model, as shown in Figure 4.1.4.

```
y_pred = model.predict(X_test)
plotcm_classification_report(y_pred, y_test)
```

Figure 4.1.3 Prediction with Trained Model and Evaluation



	precision	recall	f1-score	support
0	0.90	0.70	0.79	100
1	0.61	0.85	0.71	54
accuracy			0.75	154
macro avg	0.75	0.78	0.75	154
weighted avg	0.79	0.75	0.76	154

Figure 4.1.4 Performance Report for Baseline Logistic Regression Model

Based on the confusion matrix, there are 100 non-diabetic samples and 54 diabetic samples, resulting in a total of 154 samples.

- 70 True Negative: Diabetic negative individual correctly predicted as diabetic negative,

- 30 False Positive: Diabetic negative individual wrongly predicted as diabetic positive,
- 8 False Negative: Diabetic positive individual wrongly predicted as diabetic negative,
- 46 True Positive: Diabetic positive individual correctly predicted as diabetic positive.

This confusion matrix indicates that the baseline Logistic Regression was better at detecting diabetics individual, correctly identifying 46 cases while missing only 8. However, this sensitivity came at the cost of a high false positive rate: approximately 30% of non-diabetic individuals were incorrectly classified as diabetic. These misclassifications could lead to unnecessary additional checkups, highlighting the inherent trade-off between recall and precision in the model's performance.

Moreover, given the imbalance nature of the dataset, with fewer positive cases against a larger number of negatives cases, accuracy alone is not a reliable metric, as it was potentially skewed by the majority class. Therefore, with an emphasis on the capability to detect positive cases, a binary F1 score was selected as the main evaluation metric to evaluate the ability to correctly identify diabetic cases while minimising false positives, striking a necessary balance needed in medical datasets such as the Pima Indians dataset.

Based on the classification report in Figure 4.1.4, the model achieved a binary F1 score of 0.71 for the positive class (1). However, when compared to the F1 score of the majority class (0) with 0.79, there are 8% difference between the F1 scores reflects the impact of class imbalance, where the majority class benefits from more training samples. This effect persists even with the *class_weight="balanced"* parameter that was applied during training, suggesting that balancing weights only partially mitigates the influence of unequal dataset sizes. The model remains sensitive to noise within the minority class, which limits its overall precision.

Technique: Validate baseline model with 5-Fold Cross-Validation

To verify the reliability of the model and ensure the results in the confusion matrix and classification report are honest, 5-Fold Cross-Validation (5CV) was performed. By using F1-scoring on the positive class, the model's performance was evaluated across five distinct subsets of the training data. This process provides a "Mean CV F1" score, which serves as a more honest representation of how the model will generalise to unseen samples.

```python
cv_scores = cross_val_score(model, X_train, y_train, cv=5, scoring='f1')

print(f"Cross-Validation F1 Scores: {cv_scores}")
print(f"Mean CV F1: {cv_scores.mean():.4f}")
print(f"Standard Deviation of CV F1: {cv_scores.std():.4f}")
```

Figure 4.1.5 Code for 5-Fold Cross-Validation

After performing cross-validation, we obtain the result in Figure 4.1.6.

```
Cross-Validation F1 Scores: [0.78723404 0.68965517 0.71111111 0.7        0.60606061]
Mean CV F1: 0.6988
Standard Deviation of CV F1: 0.0577
```

Figure 4.1.6 5-fold cross-validation for Baseline Logistic Regression Model

Based on the classification report's F1-score of 0.71, the model demonstrates strong localised performance on the first fold with a score of ~0.79 (+0.08). While folds two to four align closely with the test set's baseline, the fifth fold underperforms significantly at ~0.61 (-0.10), indicating a potential sensitivity to specific data distributions or outliers within that fifth fold subset. This variance naturally increases the standard deviation of the validation results; however, the Mean CV F1 score of ~0.70 remains close to the test set performance. Thus, the cross-validation results are still considered material and statistically relevant for performance comparisons.

Technique: Fine-tuning Logistic Regression and evaluating performance

In preparation for model comparisons, each model in Sections 4.1 and 4.2 was fine-tuned to compare the peak performance of each model for this specific dataset with the same preprocessing methods.

For Logistic Regression, the fine-tuning process used GridSearchCV with five-fold cross-validation to identify the optimal hyperparameter configuration. The parameter grid used is shown in Figure 4.1.7.

```
param_grid = [
    {
        'penalty': ['elasticnet'],
        'solver': ['saga'],
        'l1_ratio': [0, 0.5, 1],
        'C': [0.01, 0.1, 1.0, 10.0],
        'class_weight': ['balanced', None]
    },
    {
        'solver': ['lbfgs', 'liblinear', 'saga'],
        'C': [0.01, 0.1, 0.5, 1.0, 10.0],
        'class_weight': ['balanced', None]
    },
]
```

Figure 4.1.7 Parameter Grid for Tuning Logistic Regression Model

Table 4.1.1 defines the parameters.

Table 4.1.1 Logistic Regression Model Fine-Tuning Parameters Definition

Parameter	Description
penalty	Defines the type of regularisation applied to the model coefficients (e.g. L1, L2, or Elastic Net)
solver	Algorithm used to optimise the Logistic Regression objective (e.g. lbfgs, liblinear, saga)
l1_ratio	Controls the mix between L1 and L2 regularisation when using Elastic Net (0 = L2, 1 = L1, 0-1 = blend)
C	Inverse of regularisation strength; smaller values mean stronger regularisation; larger values mean more complex models
class_weight	Adjusts weights assigned to classes during training; balanced compensates for class imbalance, while None applies no adjustment

Next, a GridSearchCV was initialised with the function in Figure 4.1.8 and was fitted with the training set.

```
grid_search = GridSearchCV(
    estimator=LogisticRegression(max_iter=1000),
    param_grid=param_grid,
    cv=5,
    scoring=['f1']
    refit='f1'
    )
grid_search.fit(X_train, y_train)

print(f"\nBest Parameters Found: {grid_search.best_params_}")
```

Figure 4.1.8 GridSearchCV Initialisation

Table 4.1.2 defines the parameters.

Table 4.1.2 GridSearchCV Parameters Definition

Parameter	**Description**
estimator	The estimator is the Logistic Regression classifier to be tuned, with max_iter set to 1000 to ensure convergence
param_grid	The defined parameter grid in Figure 4.1.5 created is supplied here to explore different combinations of hyperparameters
CV	Set to 5 for five-fold cross-validation, ensuring robust validation of the best model
scoring	Specifies the evaluation metric, which is F1 for positive class
refit	Indicate which scoring metric should be used to refit the model on the entire training set after the best parameters are selected

After fitting the model and searching for the best hyperparameter combination, the output from the GridSearchCV procedure (shown in Figure 4.1.9) displays the selected parameters. This configuration represents the optimal balance of regularisation, solver choice, and class weighting that produced the highest F1 score (for the positive class) under five-fold cross-validation. By relying on cross-validation rather than a single train-test split, the selected parameters provide a more reliable estimate of generalisation performance and ensure that the model comparison in subsequent sections was honest.

```
Best Parameters Found: {'C': 0.1, 'class_weight': 'balanced', 'solver': 'liblinear'}
```

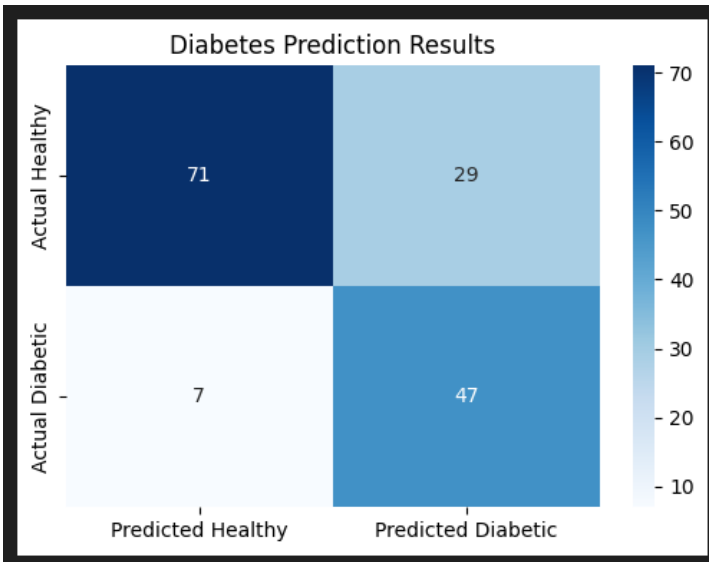
Figure 4.1.9 Best Parameter Found for Logistic Regression Model

To demonstrate the reliability of the selected hyperparameter combination, the refitted model obtained from *grid_search.best_estimator_* was trained on the full training set and subsequently used to generate predictions on the test set as shown in Figure 4.1.10.

```
best_model = grid_search.best_estimator_
y_pred_tuned = best_model.predict(X_test)
```

Figure 4.1.10 Retrieving Best F1 Score

The resulting performance is illustrated in Figure 4.1.11. This evaluation provides an unbiased measure of the tuned model's generalisation capability and confirms that the chosen parameters yield the best balance of precision and recall under cross-validation.



	precision	recall	f1-score	support
0	0.91	0.71	0.80	100
1	0.62	0.87	0.72	54
accuracy			0.77	154
macro avg	0.76	0.79	0.76	154
weighted avg	0.81	0.77	0.77	154

Figure 4.1.11 Performance Report for Fine-Tuned Logistic Regression Model

Based on the confusion matrix, the predictions improved compared to the base model as follows:

- True negatives increased by to 71 (+1), while false positives decreased by to 29 (-1).
- False negatives decreased to 7 (-1), while true positives increased to 47 (+1).

The baseline model achieved an F1 score of 0.71 for the positive class and 0.79 for the negative class with an 8% gap. In comparison, the fine-tuned model provided a targeted improvement by correctly identifying 47 diabetic cases (reducing false negatives by 1) and correctly classifying 71 healthy individuals (reducing false positives by 1). This dual reduction in classification errors improved the model's F1 scores to 0.72 for the positive class and 0.80 for the negative class. Most notably, this fine-tuning successfully narrowed the F1 performance gap between the positive and negative classes from 8% to 7%, demonstrating a slightly better mitigation against class imbalance and yielding a better overall balance of precision and recall.

4.2 Comparative Model (Random Forest & XGBoost Model)

To benchmark the performance of Logistic Regression, two additional binary classification models, Random Forest and XGBoost, were fine-tuned using the same dataset and preprocessing pipeline to ensure consistency. Hyperparameter optimisation was conducted using grid search with similar 5-fold cross-validation, ensuring each model reached its peak performance for a fair comparison.

Random Forest: Implementation

Random Forest is an ensemble learning method that builds numerous decision trees and combines their results to improve accuracy and control overfitting. While Logistic Regression calculates a linear relationship between variables, Random Forest relies on a "forest" of randomised trees to capture more complex, non-linear patterns in biological data.

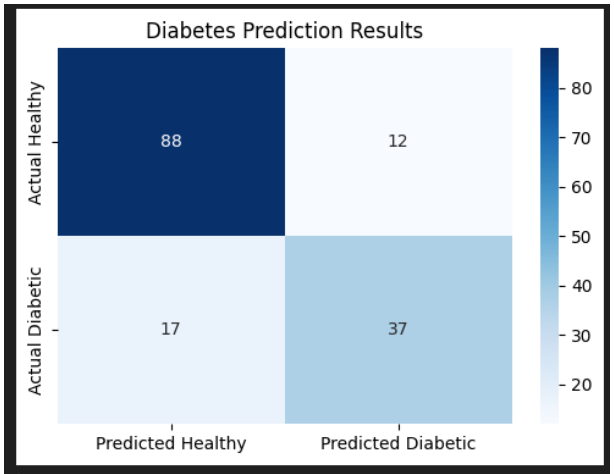
The Random Forest classifier was integrated into the same Scikit-Learn pipeline as the Logistic Regression model, replacing *LogisticRegression()* with *RandomForestClassifier()* as shown in Figure 4.2.1. Unlike Logistic Regression, which uses *max_iter* for *solver* convergence, Random Forest is an ensemble of trees that does not require iteration. A *random_state* was implemented to ensure the stochastic bootstrap sampling and feature selection remain reproducible.

```
model = RandomForestClassifier(class_weight="balanced", random_state=42)
model.fit(X_train, y_train)
```

Figure 4.2.1 Code for Training Baseline Random Forest

Random Forest: Baseline Performance and Statistical Validation

The baseline performance metrics are captured in Figure 4.2.2, with the statistical reliability verified via 5-fold cross-validation in Figure 4.2.3.



	precision	recall	f1-score	support
0	0.84	0.88	0.86	100
1	0.76	0.69	0.72	54
accuracy			0.81	154
macro avg	0.80	0.78	0.79	154
weighted avg	0.81	0.81	0.81	154

Figure 4.2.2 Performance Report for Baseline Random Forest Model

```
Cross-Validation F1 Scores: [0.60526316 0.60526316 0.58666667 0.62068966 0.60465116]
Mean CV F1: 0.6045
Standard Deviation of CV F1: 0.0108
```

Figure 4.2.3 5-Fold Cross-Validation for baseline Random Forest model

A significant discrepancy is observed between the test set's F1-score (0.72) and the 5-fold cross-validation (5CV) mean F1-score (0.60). These +0.12 elevations in test performance suggest that the specific test partition may not be fully representative of the broader data distribution.

Crucially, the 5CV process yielded highly consistent results, with a mean of 0.6045 and an exceptionally low standard deviation of 0.0108. This minimal variance across all five folds

indicates that the model has reached a stable convergence; however, it also confirms that the model's true predictive power was likely centred around 0.60 rather than the optimistic 0.72 seen in the classification report. So, while the test results are encouraging, the 5CV F1 scoring provides a more statistically honest baseline for evaluating the model's real-world generalisation. Therefore, comparing the fine-tuned model against this baseline's test performance may not provide an accurate representation of the model's true performance.

Random Forest: Model Tuning and Configuration

To optimise the model, a hyperparameter grid was constructed to target the forest's complexity and regularisation, as shown in Figure 4.2.4.

```
param_grid = {
    'class_weight': [None, 'balanced', 'balanced_subsample'],
    'n_estimators': [100, 200, 300, 400, 500],
    'max_depth': [None, 5, 6, 7, 8],
    'min_samples_split': [2, 14, 15, 16],
}
```

Figure 4.2.4 Parameter grid for Random Forest Model

The optimisation process is as defined in Table 4.2.1.

Table 4.2.1 Random Forest Model Fine-tuning Parameters Definition

Parameter	Description
class_weight	Adjusts weights assigned to classes during training: *balanced* compensates for class imbalance, *balanced_subsample* calculates weights per bootstrap sample to handle imbalance, while *None* applies no adjustment
n_estimators	The numbers of tree for the prediction. A higher count generally increases the model stability and performance at the cost of higher computational power and memory.
max_depth	Limits tree levels to prevent overfitting by restricting the complexity of individual trees.

min_samples_split The minimum samples required to split a node. Higher values act as regularisation to prevent the model from learning noise.

The optimisation was executed using GridSearchCV with F1-scoring to prioritise the minority class (Figure 4.2.5). The resulting fine-tuned model's best parameter that yielded the most stable performance is documented in Figure 4.2.6.

```
grid_search = GridSearchCV(
    estimator=RandomForestClassifier(random_state=42),
    param_grid=param_grid,
    cv=5,
    scoring='f1',
    n_jobs=-1
)
grid_search.fit(X_train, y_train)
```

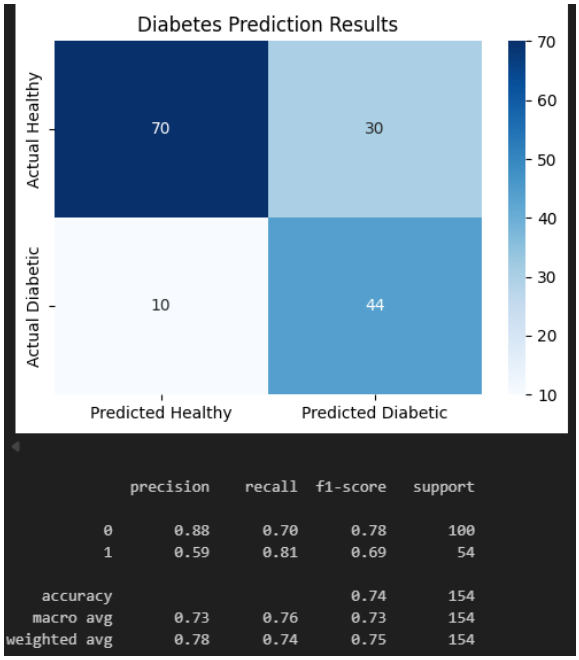
Figure 4.2.5 Initialising GridSearchCV for Random Forest Model

```
Best Parameters Found: {'class_weight': 'balanced_subsample', 'max_depth': 7, 'min_samples_split': 15, 'n_estimators': 500}
```

Figure 4.2.6 Optimised Hyperparameters for Random Forest

Random Forest: Base and Fine-Tuned Model Performance Evaluation

The performance of the fine-tuned Random Forest model was evaluated using a confusion matrix and classification report, as illustrated in Figure 4.2.7.



	precision	recall	f1-score	support
0	0.88	0.70	0.78	100
1	0.59	0.81	0.69	54
accuracy			0.74	154
macro avg	0.73	0.76	0.73	154
weighted avg	0.78	0.74	0.75	154

Figure 4.2.7 Performance Report for Fine-Tuned Random Forest Model

Given the class imbalance of the dataset, the model's performance was evaluated based on the positive class (1) from the classification report.

Firstly, the model demonstrated a strong Recall of 0.81 for the positive class, successfully identifying 44 out of 54 actual diabetic cases. This indicates that the model was highly sensitive, which is a critical requirement in healthcare diagnostics to ensure fewer cases go undetected. However, the model achieved a Precision of 0.59, resulting in a substantial 30 False Positives. This lower precision was a deliberate trade-off, likely resulting from the *class_weight* optimisation to prioritise finding all positive cases with a false positive trade-off (sensitivity over specificity and precision).

Consequently, this results in an F1-score of 0.69 for the positive class. While this score was lower than the majority class F1 (0.78), it represents a statistically balanced performance for detecting the diabetic positive class. In a medical context, this configuration was considered as the "cost" of a False Positive was significantly lower than the potential life-threatening cost of failing to identify a diabetic positive individual.

XGBoost Classifier: Implementation

XGBoost is an optimised distributed gradient boosting library designed to be highly efficient and flexible. Unlike Random Forest, which builds independent trees in parallel, XGBoost builds trees sequentially, where each new tree attempts to correct the errors of the previous ones.

As shown in Figure 4.2.8, the model specifically addressed the dataset's class imbalance by calculating the *scale_pos_weight* similar to the Scikit-Learn pipeline's *class_weight='balance'* parameter. By dividing the total number of negative samples by positive samples, the model assigns higher importance to the diabetic positive class during the gradient boosting process.

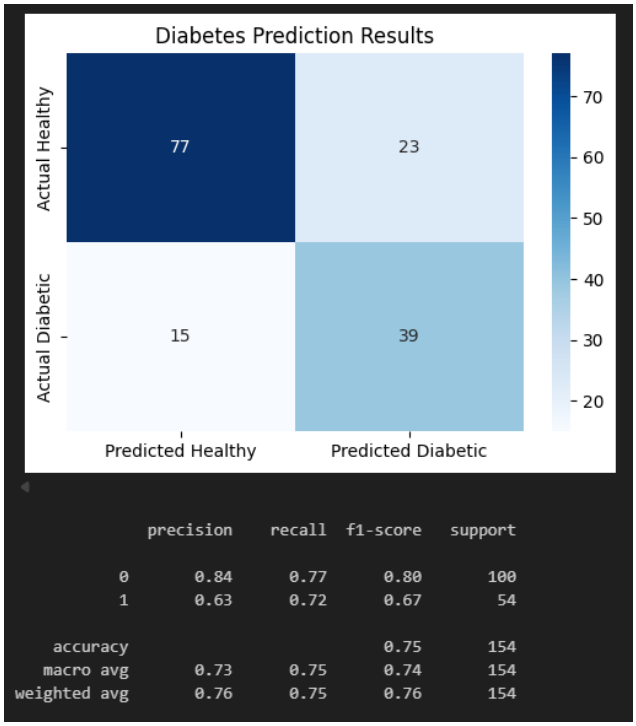
```
scale_pos_weight = np.sum(y_train == 0) / np.sum(y_train == 1)
print(f"Scale Pos Weight (Balance): {scale_pos_weight:.2f}")

model = XGBClassifier(
    eval_metric='logloss',
    scale_pos_weight=scale_pos_weight,
    random_state=42,
    tree_method='hist'
)
model.fit(X_train, y_train)
```

Figure 4.2.8 Initialising Baseline XGBoost Model with Class Weighting

XGBoost Classifier: Baseline Performance and Statistical Validation

The baseline performance was captured in Figure 4.2.9, while the statistical reliability was verified using 5-fold cross-validation in Figure 4.2.10.



	precision	recall	f1-score	support
0	0.84	0.77	0.80	100
1	0.63	0.72	0.67	54
accuracy			0.75	154
macro avg	0.73	0.75	0.74	154
weighted avg	0.76	0.75	0.76	154

Figure 4.2.9 Class Weight Value and Performance Report for Baseline XGBoost Model

```
Cross-Validation F1 Scores: [0.65934066 0.68235294 0.575      0.66666667 0.60416667]
Mean CV F1: 0.6375
Standard Deviation of CV F1: 0.0409
```

Figure 4.2.10 5-Fold Cross-Validation for Baseline XGBoost Model

In contrast to the Random Forest base model, the XGBoost baseline exhibited high generalisation stability. The test set achieved an F1-score of 0.67, closely aligned with the 5-fold cross-validation mean of 0.6375. The narrow discrepancy of +0.0325 fell well within the standard deviation of 0.0409, indicating consistent model performance across folds. The XGBoost model demonstrates that its performance is not a result of "test-set luck"; it reflects a consistent ability to capture underlying patterns across multiple data folds. This makes the baseline XGBoost model a highly honest starting point for further optimisation.

XGBoost Classifier: Model Tuning and Configuration

To further refine the model, a hyperparameter grid was constructed as shown in Figure 4.2.11. The tuning focused on the learning rate and tree-specific parameters to prevent the sequential boosting process from overfitting.

```
param_grid = {
    'learning_rate': [1e-3, 0.01, 0.1, 0.5, 1],
    'max_depth': [3, 4, 5, 6, 7],
    'n_estimators': [100, 200, 300, 400],
    'subsample': [0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9],
    'colsample_bytree': [0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9]
}
```

Figure 4.2.11 Hyperparameter Grid for XGBoost Optimisation

Table 4.2.2 details the fine-tuning hyperparameters for the XGBoost Model.

Table 4.2.2 XGBoost Model Fine-Tuning Parameters Definition

Parameter	**Description**
learning_rate	It shrinks the feature weights after each boosting step to make the boosting process more conservative and prevent overfitting.
max_depth	Determines how deeply each individual tree can grow. Deeper trees capture more information but increase the risk of learning noise.
n_estimators	The total number of boosting rounds (or trees) to be run.
subsample	The fraction of observations to be randomly sampled for each tree. Lower values prevent overfitting.
colsample_bytree	The fraction of columns (features) to be randomly sampled for each tree.

The optimisation was executed with GridSearchCV using F1-scoring to prioritise the minority class, as detailed in Figure 4.2.12. The resulting fine-tuned model parameters are documented in Figure 4.2.13.

```
grid_search = GridSearchCV(
    estimator=XGBClassifier(
        eval_metric='logloss',
        scale_pos_weight=scale_pos_weight,
        random_state=42
    ),
    param_grid=param_grid,
    cv=5,
    scoring='f1',
    n_jobs=-1
)
```

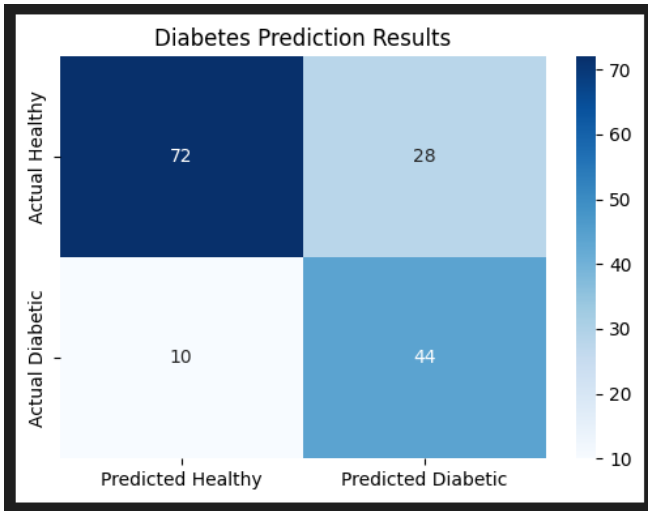
Figure 4.2.12 Initialising GridSearchCV for XGBoost Model

```
Best Parameters: {'colsample_bytree': 0.2, 'learning_rate': 0.01, 'max_depth': 5, 'n_estimators': 200, 'subsample': 0.3}
```

Figure 4.2.13 Optimised Hyperparameters for XGBoost Model

XGBoost Classifier: Base and Fine-Tuned Model Performance Evaluation

The performance of the optimised XGBoost model was illustrated in Figure 4.2.14. Following the same pipeline and techniques, the evaluation centres on the diabetic positive class (1).



	precision	recall	f1-score	support
0	0.88	0.72	0.79	100
1	0.61	0.81	0.70	54
accuracy			0.75	154
macro avg	0.74	0.77	0.74	154
weighted avg	0.78	0.75	0.76	154

Figure 4.2.14 Performance Report for Fine-Tuned XGBoost Model

The fine-tuned XGBoost model achieved a positive class F1-score of 0.70, reflecting a meaningful improvement from the baseline of 0.67 and indicating a more effective balance between precision and recall following optimisation. A key strength of the model is its high sensitivity, maintaining a strong recall of 0.81 by successfully identifying 44 out of 54 actual diabetic cases. This makes sure the model remains a reliable screening tool with a high detection rate essential for clinical utility. While precision improved slightly to 0.61, it remains a deliberate trade-off where sensitivity was prioritised over specificity and precision. By capturing 81% of diabetic cases while misclassifying only 28 healthy individuals as positive, the tuned XGBoost model establishes a robust and conservative diagnostic profile suitable for early detection.

CHAPTER 5 COMPARATIVE ANALYSIS

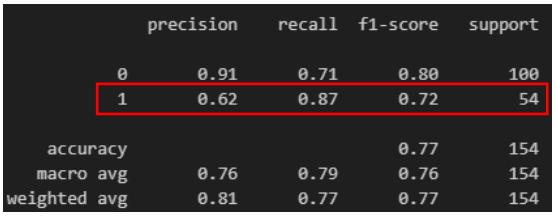
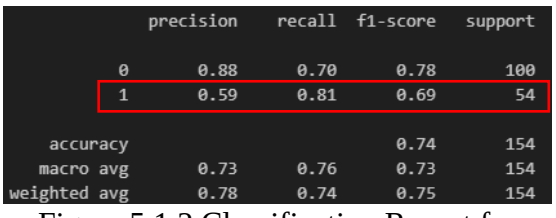
This chapter presents a systematic evaluation and comparison of the machine learning models developed for diabetes prediction in Chapter 4. This analysis aims to identify the best-performing model for the Pima Indian Diabetes dataset by assessing performance through a multi-dimensional framework. This includes class-specific metrics, error distribution, ROC curves, AUC scores, and bias-variance trade-off.

5.1 Confusion Matrix and Classification Report Analysis

Table 5.1.1 represents the performance metrics and classification reports for the fine-tuned Logistic Regression, Random Forest, and XGBoost models developed in Chapter 4.

Due to the inherent imbalance in the dataset, our primary focus was on correctly identifying diabetic patients (Class 1: Positive Class). Therefore, rather than relying solely on overall accuracy, sensitivity, or specificity, this evaluation will prioritise the Precision, Recall, and F1-score of Class 1, as highlighted with red-outlined box. Additionally, True Negatives (Actual: 0, Predicted: 0) in the confusion matrix hold less weight in our final model selection criteria compared to how well the models detect true diabetic cases.

Table 5.1.1 Performance Summary for Fine-Tuned Models

<table>
<tr><th>Confusion matrix
0 - Diabetic Negative (Healthy)
1 - Diabetic Positive</th><th>Classification Report</th></tr>
<tr><td>Logistic Regression
<table>
<tr><th>Predicted / Actual</th><th>0</th><th>1</th></tr>
<tr><td>0</td><td>71</td><td>29</td></tr>
<tr><td>1</td><td>7</td><td>47</td></tr>
</table></td><td> 
<table>
<tr><th></th><th>precision</th><th>recall</th><th>f1-score</th><th>support</th></tr>
<tr><td>0</td><td>0.91</td><td>0.71</td><td>0.80</td><td>100</td></tr>
<tr><td>1</td><td>0.62</td><td>0.87</td><td>0.72</td><td>54</td></tr>
<tr><td>accuracy</td><td></td><td></td><td>0.77</td><td>154</td></tr>
<tr><td>macro avg</td><td>0.76</td><td>0.79</td><td>0.76</td><td>154</td></tr>
<tr><td>weighted avg</td><td>0.81</td><td>0.77</td><td>0.77</td><td>154</td></tr>
</table>
Figure 5.1.1 Classification Report for Logistic Regression Model</td></tr>
<tr><td>Random Forest
<table>
<tr><th>Predicted / Actual</th><th>0</th><th>1</th></tr>
<tr><td>0</td><td>70</td><td>30</td></tr>
<tr><td>1</td><td>10</td><td>44</td></tr>
</table></td><td> 
<table>
<tr><th></th><th>precision</th><th>recall</th><th>f1-score</th><th>support</th></tr>
<tr><td>0</td><td>0.88</td><td>0.70</td><td>0.78</td><td>100</td></tr>
<tr><td>1</td><td>0.59</td><td>0.81</td><td>0.69</td><td>54</td></tr>
<tr><td>accuracy</td><td></td><td></td><td>0.74</td><td>154</td></tr>
<tr><td>macro avg</td><td>0.73</td><td>0.76</td><td>0.73</td><td>154</td></tr>
<tr><td>weighted avg</td><td>0.78</td><td>0.74</td><td>0.75</td><td>154</td></tr>
</table>
Figure 5.1.2 Classification Report for Random Forest Model</td></tr>
</table>

XGBoost

Actual \ Predicted	0	1
0	72	28
1	10	44

	precision	recall	f1-score	support
0	0.88	0.72	0.79	100
1	0.61	0.81	0.70	54
accuracy			0.75	154
macro avg	0.74	0.77	0.74	154
weighted avg	0.78	0.75	0.76	154

Figure 5.1.3 Classification Report for XGBoost Model

Precision

Precision measures the reliability of the model's correct predictions, the percentage of patients predicted as diabetic who actually have the disease. Due to the imbalanced dataset and the small number of class 1 cases, precision tends to drop. Despite this, Logistic Regression achieved the highest precision at 62%. While XGBoost produced slightly fewer false positives overall (28 compared to Logistic Regression's 29), Logistic Regression's ability to correctly identify significantly more True Positives (47 compared to 44) resulted in a stronger overall precision ratio. Random Forest performed the poorest in this metric at 0.59.

Recall

In medical screening, like diabetes prediction, Recall is often considered the most critical metric. It measures the model's ability to correctly identify all actual diabetic patients, minimising False Negatives. Missing a diagnosis carries a high risk, as late discovery can increase severity. Based on the classification report for Class 1, Logistic Regression significantly outperforms the other models, achieving the highest recall of 87%. It successfully identified 47 out of 54 actual positive cases, missing only 7 cases. In contrast, both Random Forest and XGBoost achieved a lower recall of 81%, where each missed 10 positive cases.

F1-Score & Overall Misclassifications

The F1-score serves as the harmonic mean between Precision and Recall, providing a single metric to evaluate the model's performance on the minority class. Because Logistic Regression outperformed the tree-based models in both identifying true cases (Recall) and maintaining prediction reliability (Precision), it achieved the highest F1-score of 0.72. XGBoost and Random Forest fall behind with 0.70 and 0.69, respectively.

Furthermore, analysing the confusion matrices as a whole, Logistic Regression demonstrates the lowest overall number of misclassifications, with a total of 36 errors across the test set. In comparison, XGBoost and Random Forest recorded 38 and 40 misclassifications, respectively.

5.2 ROC-AUC Performance Comparisons

While metrics like Precision, Recall, and F1-Score evaluate a model's performance at a specific, fixed threshold (0.5), the Receiver Operating Characteristic Area Under the Curve (ROC-AUC) provides a broader, aggregate measure of performance. It evaluates the model's underlying ability to distinguish between the two classes: diabetic patients (Class 1) and healthy individuals (Class 0) across all possible thresholds. An AUC score closer to 1.0 indicates an excellent model, while a score near 0.5 (equivalent to the ROC line getting nearer to the diagonal line) suggests the model is random guessing.

Looking at the overall diagnostic ability of the models in Figure 5.2.1, all three classifiers demonstrated strong representation of the dataset, each scoring above 0.85. Consistent with the findings in Section 5.1, Logistic Regression outperform Random Forest and XGBoost models again, achieving the highest ROC-AUC score of 86.8%. This suggests that Logistic Regression produces well-calibrated probability distributions, enabling reliable discrimination between positive and negative diabetes cases across a range of decision thresholds.

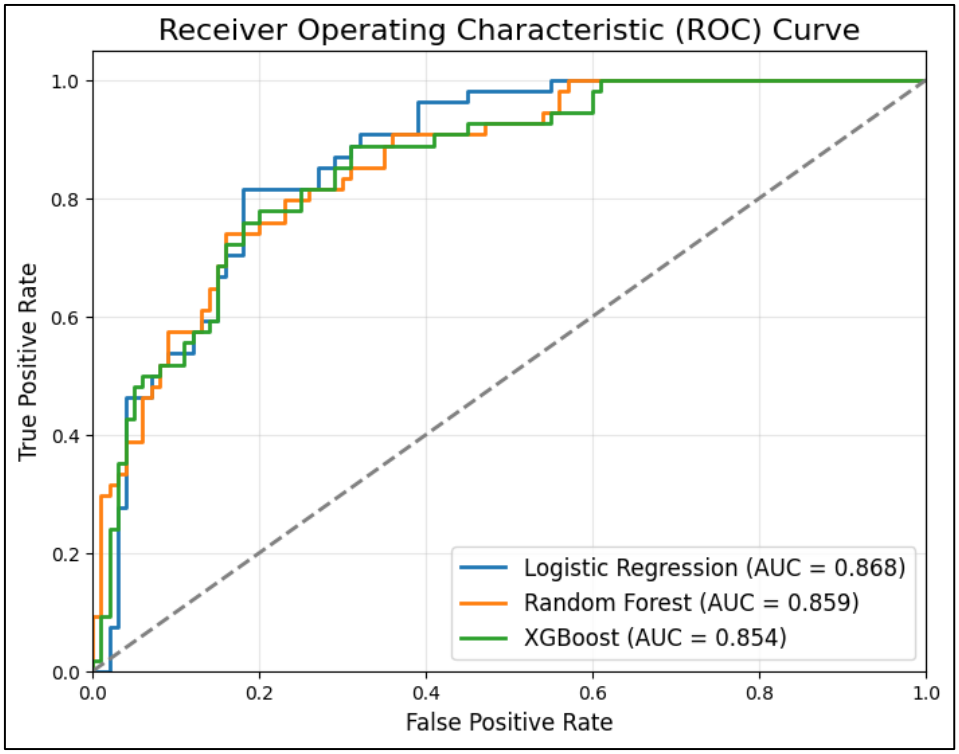


Figure 5.2.1 ROC-AUC Curve

Random Forest followed closely behind with an AUC of 85.8%, while XGBoost recorded the lowest score of the three at 85.4%. The fact that the simpler, linear Logistic Regression model outperformed the more complex, non-linear ensemble methods like Random Forest and XGBoost in overall class separation further validates that with proper, clinically backed feature engineering, a simple model such as Linear Regression can perform well.

5.3 Precision-Recall AUC (PR-AUC) Curve

Because the Pima Indian Diabetes dataset suffers from class imbalance, evaluating the models using the ROC-AUC can possibly present an overly confident view of performance. As a verification, a Precision-Recall (PR) Curve was utilised. The PR curve focuses exclusively on the performance of the minority class (Class 1) by plotting Precision against Recall on all possible classification thresholds.

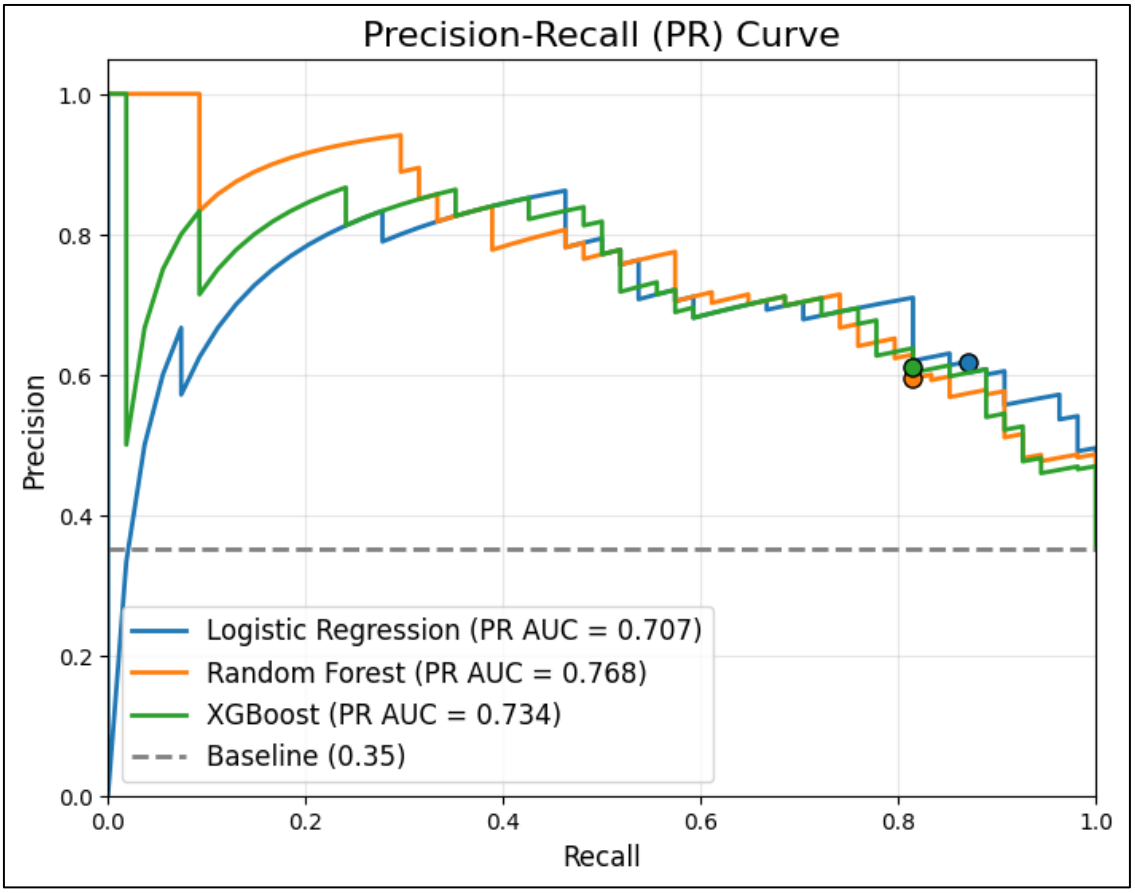


Figure 5.3.1 Precision-Recall (PR) Curve

As illustrated in Figure 5.3.1, all three models perform significantly better than the random guessing baseline of 35% (the ratio of Class 1 test samples to total test samples). The markers on the graph indicate exactly where each model's performance sits at the default 0.5 decision threshold. At this specific default boundary, Logistic Regression (blue line) captures the most optimal balance between Precision and Recall that leads to the highest F1 score. Its default position favours Recall over Precision, which is clinically important.

When considering total PR AUC, however, Random Forest leads at 76.8%, followed by XGBoost at 73.4%, with Logistic Regression at 70.7%. This reversal was driven by tree-based models starting with a perfect Precision threshold (~1.00), which inflates their AUC. Conversely, Logistic Regression begins at 0.0 Precision, lowering its overall area despite strong F1 performance.

Even with this penalty, Logistic Regression remains the most practical model for screening. Performance at near-zero Recall but with extremely good precision is clinically irrelevant, as detecting only one or two patients practically offers no real assessment. In the critical region of

the PR Curve where Recall exceeds ~0.70, Logistic Regression matches or surpasses both tree-based models in precision, offering more room to achieve a robust PR balance for real-world patient detection.

5.4 F1 Threshold Optimisation and Comparative Analysis

Because the default decision boundary of 0.5 does not inherently account for data imbalance, a targeted threshold optimisation technique was applied to all three models. To achieve this without introducing data leakage from the test set, the following function was utilised to find an optimal decision threshold using out-of-fold cross-validation on the training set only, as shown in Figure 5.4.1.

```
def get_optimal_f1_threshold(model, X_train, y_train, cv=5):
    from sklearn.model_selection import StratifiedKFold

    best_thresh = 0
    best_f1 = 0

    skf = StratifiedKFold(n_splits=cv)
    for threshold in np.arange(0.1, 0.9, 0.01):
        f1s = []
        for _train_idx, val_idx in skf.split(X_train, y_train):
            X_val = X_train.iloc[val_idx]
            y_val = y_train[val_idx]
            y_probs = model.predict_proba(X_val)[:, 1]
            y_guess = (y_probs >= threshold).astype(int)
            f1s.append(f1_score(y_val, y_guess))
        if np.mean(f1s) > best_f1:
            best_f1 = np.mean(f1s)
            best_thresh = threshold
    return best_thresh
```

Figure 5.4.1 Function for Optimising Threshold for F1

This function thoroughly evaluates 80 different classification thresholds (from 0.10 to 0.89). At each step, it performs Stratified 5-Fold Cross-Validation, calculates the average F1-Score across the validation folds, and returns the threshold that produced the absolute highest validation F1-Score.

These optimised boundaries were then applied to the unseen test set to determine if shifting the classification threshold successfully improved real-world predictive performance. Table 5.4.1 details these aggregated metrics, comparing the baseline 0.5 threshold against the out-of-fold CV-optimised boundaries.

Figure 5.4.2 Performance Summary

	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
0	Logistic Regression (Threshold 0.5)	0.766234	0.618421	0.870370	0.723077	0.868148
1	Logistic Regression (Optimal Thresh: 0.52)	0.766234	0.628571	0.814815	0.709677	0.868148
2	Random Forest (Threshold 0.5)	0.740260	0.594595	0.814815	0.687500	0.858704
3	Random Forest (Optimal Thresh: 0.54)	0.772727	0.646154	0.777778	0.705882	0.858704
4	XGBoost (Threshold 0.5)	0.753247	0.611111	0.814815	0.698413	0.854444
5	XGBoost (Optimal Thresh: 0.50)	0.753247	0.611111	0.814815	0.698413	0.854444

Random Forest Evaluation

Consistent with its broad underlying area on the PR Curve in Figure 5.3.1, threshold optimisation proved highly effective for the Random Forest architecture. The algorithm identified a stricter boundary of 0.54. When applied to the Test Set, this threshold successfully filtered out false positives. As a result, the precision was boosted from 59.4% to 64.6% with an acceptable penalty to recall (-3.7% against the fine-tuned model). Consequently, the test F1-score improved from 0.687 to 0.705, and overall accuracy rose to 77.2%. However, in the context of determining the diabetic patient, lowering the recall could cause higher False Negative or more patient not being detected.

Logistic Regression Evaluation

Conversely, threshold optimisation hindered the Logistic Regression model. The cross-validation process suggested a threshold of 0.52. However, when applied to the unseen test set, the stricter boundary caused the model to miss several true positive cases, dropping recall significantly from 87.0% down to 81.4%. As a result, the test F1-score dropped from 0.723 to 0.709. This indicates that the optimised threshold slightly overfitted to the training splits, and the default 0.5 boundary remains superior for generalising.

XGBoost Evaluation

The cross-validation process determined that the optimal threshold was exactly 0.50. Therefore, the performance metrics remain identical.

While threshold tuning improved the Random Forest model's predictive capability on unseen data, it did not surpass the baseline performance of the fine-tuned linear model, Logistic Regression. Even without optimisation, Logistic Regression at the default 0.5 threshold achieved the highest F1-Score 72.3% and the highest Recall 87.0% across all classifiers and thresholds. Since maximising the detection of actual diabetic patients is the clinically critical objective, Logistic Regression was clearly the best-performing architecture.

5.5 Bias-Variance Trade Off and Discussion

To contextualise the performance of the fine-tuned algorithms, a Bias-Variance trade-off analysis was conducted. Understanding whether a model is suffering from high bias (underfitting) or high variance (overfitting) is critical for verifying its reliability on unseen medical data. To visualise this, Learning Curves were generated to compare the Training F1-Scores against the Cross-Validation F1-Scores across increasing amounts of training data.

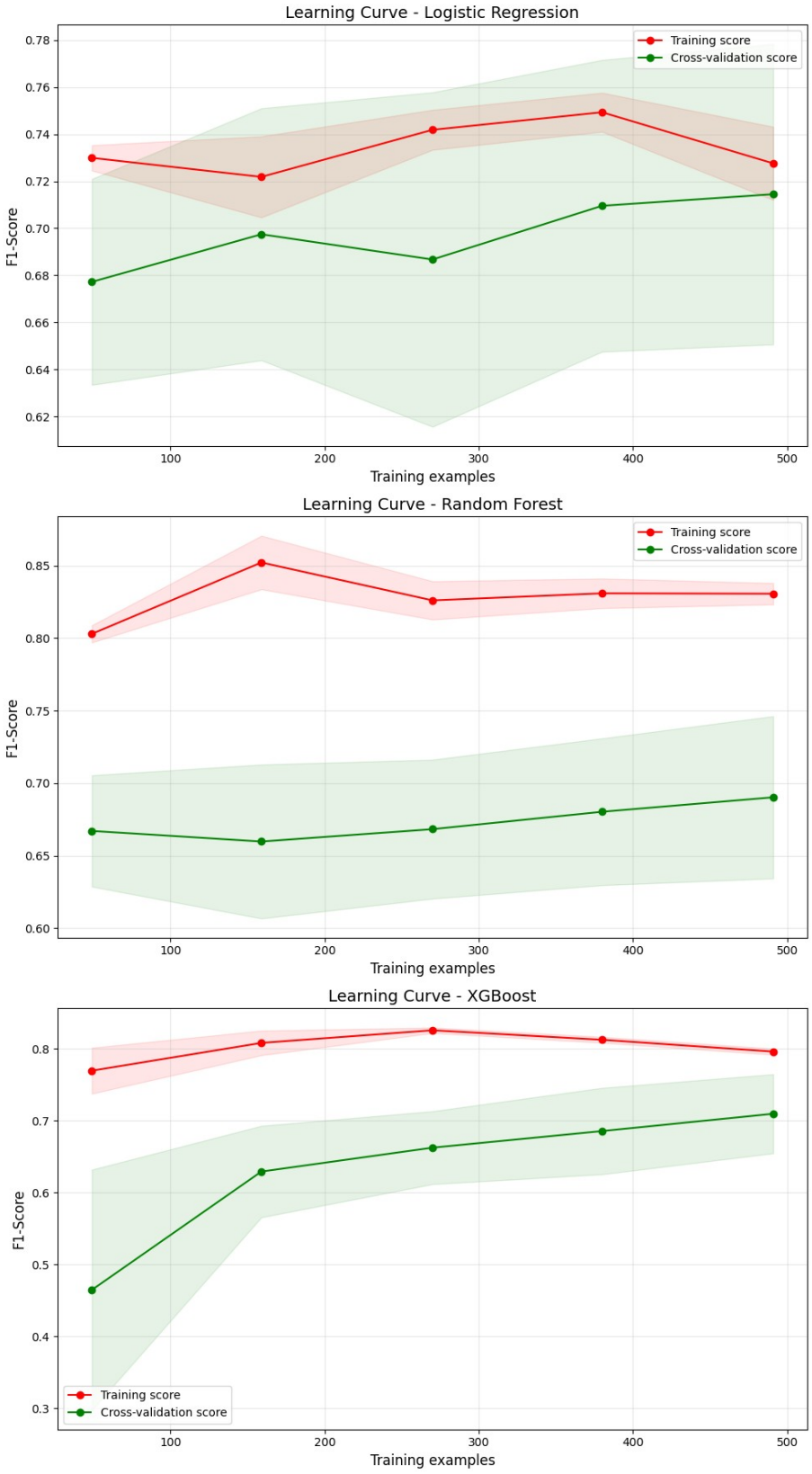


Figure 5.5.1 Learning Curves

As depicted in the Learning Curves, a distinct contrast in learning behaviour exists between the linear model (Logistic Regression) and the tree-based ensemble (Random Forest & XGBooster) models.

Logistic Regression (Optimal Balance)

Conversely, Logistic Regression demonstrates an incredibly stable performance profile. As seen in its learning curve, the gap between its Training F1-Score and its CV F1-Score was remarkably small, with both lines pulling tightly together around the 0.72 mark as more data was introduced. Because the scores converge closely, Logistic Regression exhibits low variance. While it cannot perfectly classify every instance (indicating the presence of some inherent bias, or underfitting), it proves to be the most healthy and reliable model. It captures the true underlying relationship between the medical diagnostic features and the onset of diabetes without being distracted by localised noise.

Tree-Based Models (High Variance)

Both Random Forest and XGBoost exhibit high variance. For example, as seen in the Random Forest learning curve, its performance on the training data climbs to over 0.85, but its performance on the cross-validation data plateaus around 0.69. This massive, unclosing gap between training and validation performance indicates that these complex, non-linear models are overfitting. They have essentially memorised the noise in the training data rather than generalising the underlying patterns of diabetes. While hyperparameter tuning mitigated this, the variance remains a structural weakness of these models on this specific dataset.

Throughout this comparative analysis, Logistic Regression consistently outperformed more complex, tree-ensemble and comparable modern models. It achieved the highest recall for diabetic patients, 87.0%, the highest overall testing F1-Score 72.3% and the strongest ROC-AUC, 86.8%. Finally, the bias-variance analysis visualises exactly why this is the case: Logistic Regression is the most structurally sound and generalisable model. Therefore, this study concludes that Logistic Regression was the best and most reliable model among the models for diagnosing diabetes using the Pima Indian Diabetes dataset.

5.6 Conclusion and Model Selection

Throughout this comparative analysis, Logistic Regression consistently outperformed more complex tree-ensembles and modern models, achieving the highest recall for diabetic patients, 87.0%, the highest overall test F1-Score 72.3% and the strong ROC-AUC 86.8%, with bias-variance analysis confirming its generalisation for this Pima Indian Diabetes dataset.

The preprocessing methodology further explains this performance gap, where ensemble models like Random Forest and XGBoost rely on identifying optimal split-points in continuous variables; the manual binning of features such as BMI, Age, and Blood Pressure restricted their native model mechanism, but at the same time benefited Logistic Regression by transforming complex non-linear relationships into linear categorical inputs. Given the inherent class imbalance and the chosen preprocessing strategy, fine-tuned Logistic Regression was clearly the most reliable classifier among all models for diagnosing diabetes using the dataset.

CHAPTER 6 CRITICAL REFLECTION

6.1 Strength and Limitation

While the developed models demonstrate high sensitivity, several constraints impact their broader applicability and real-world implementation.

- **Strengths:** The models, particularly the fine-tuned Logistic Regression, show a robust ability to prioritise class 1 (Recall 0.87), making them effective preliminary screening tools. The use of class-weighting successfully addressed the dataset's inherent imbalance.
- **Small Dataset Size:** The dataset contains only 768 records. This small sample size increases the risk of overfitting and limits the model's ability to generalise to a global population.
- **Lack of Demographic Diversity:** The data is exclusively focused on the Pima Indian population in Arizona, USA. Biological and genetic markers for diabetes can vary significantly across different ethnic groups; therefore, these models may not be accurate for Asian or European populations.
- **Data Quality and Age:** The dataset is relatively old (collected in 1990) and contains several “0” values for physiological markers, such as BMI and blood pressure, which are biologically impossible, indicating measurement inconsistencies or missing data. It may not reflect modern medical standards or current lifestyle-driven trends in diabetes management.

6.2 Ethical Consideration

The deployment of AI in healthcare requires strict consideration of ethical boundaries to prevent misinformation that could harm patients and the integrity of health systems.

- **The Impact of False Positives:** As discussed in Chapter 5, the best-performing model (Logistic Regression) achieved an F1 score with a precision of 0.61. This leads to a notable number of false positives, which may cause unnecessary psychological stress for users. Transparency about the model’s error profile is therefore essential to avoid misinterpretation or panic.

- **Algorithmic Fairness:** The imbalance in the training data could lead to biased predictions. If used in a real-world setting, a model trained on such a specific demographic might unfairly misdiagnose individuals from other backgrounds.
- **Clinical Boundaries:** It is vital to clarify that these models are intended for educational and research purposes only. They are not medical devices and must not replace the professional judgment of a healthcare provider.
- **Transparency and Data Integrity:** This study utilises an anonymised, open-source dataset from the UCI Machine Learning Repository. No personally identifiable information (PII) such as names or IDs was used, ensuring patient privacy.

6.3 Real World Implications

Despite the limitations, the study offers significant insights into the role of low-cost screening tools.

- **Preliminary Screening Tool:** These models can serve as a "first guess" digital health assistant, providing users with a quick assessment with minimal cost. This is particularly useful in resource-limited locations where clinical testing is not immediately accessible.
- **Complementary Decision Support:** The results should be viewed as a signal for the user to seek professional medical advice. By acting as a trigger for early clinical consultation, the model could contribute to earlier intervention and better long-term health outcomes.

6.4 Future Improvements

To expand this project into a viable and medically ready system, several technical and methodological enhancements are proposed.

- **Geographic and Demographic-Based Expansion:** Future work should focus on expanding the dataset to include diverse regions. For example, Malaysia. Given the high obesity rates in Malaysia, driven by local dietary habits and environmental factors, collecting "State" and "Country" data would allow for better localised feature engineering.
- **Stricter Data Collection:** Implementing mandatory fields during data collection and rejecting incomplete records would improve data integrity and reduce the reliance on null-handling techniques like KNNImputer.
- **Scalability to Deep Learning:** By increasing the dataset size to several thousand records, the research could transition from traditional machine learning to Multi-Layer Perceptron

(MLP) or other Deep Learning architectures to capture even more complex non-linear patterns.

- **Explainable AI (XAI):** Integrating tools like SHAP (SHapley Additive exPlanations) or LIME would make the model's decision-making process transparent. Understanding why a model flags a specific user (e.g., I predicted the patient is diabetic because their glucose levels and BMI are extremely high) makes the prediction more understandable and trustworthy for both patients and medical professionals.

References

Bhandari, P. (2020, August 28). *Ratio Scales | Definition, Examples, & Data Analysis*. Scribbr. https://www.scribbr.com/statistics/ratio-data/

Centers for Disease Control and Prevention. (2024, March 19). *Adult BMI Categories*. CDC. https://www.cdc.gov/bmi/adult-calculator/bmi-categories.html

Elkan, C. (2001). *The Foundations of Cost-Sensitive Learning*. https://cseweb.ucsd.edu/~elkan/rescale.pdf

IBM. (2021, October 15). *What is overfitting?* Ibm.com. https://www.ibm.com/think/topics/overfitting

International Diabetes Federation. (2025). *Diabetes Facts & Figures*. International Diabetes Federation. https://idf.org/about-diabetes/diabetes-facts-figures/

National Heart, Lung, and Blood Institute. (2024, April 25). *High blood pressure - what is high blood pressure (hypertension) | NHLBI, NIH*. Www.nhlbi.nih.gov. https://www.nhlbi.nih.gov/health/high-blood-pressure

pandas-dev. (2025). *pandas/pandas/core/nanops.py at main · pandas-dev/pandas*. GitHub. https://github.com/pandas-dev/pandas/blob/main/pandas/core/nanops.py

Scikit-Learn. (2025). *6.4. Imputation of missing values — scikit-learn 0.22.2 documentation*. Scikit-Learn.org. https://scikit-learn.org/stable/modules/impute.html

Suh, S., & Kim, J. H. (2015). Glycemic Variability: How Do We Measure It and Why Is It Important? *Diabetes & Metabolism Journal*, *39*(4), 273. https://doi.org/10.4093/dmj.2015.39.4.273

Thomas, L. (2023). *How to use stratified sampling*. Scribbr. https://www.scribbr.com/methodology/stratified-sampling/

World Health Organization. (2024, November 14). *Diabetes*. World Health Organization. https://www.who.int/news-room/fact-sheets/detail/diabetes

Yeo, I., & Johnson, R. A. (2000). A New Family of Power Transformations to Improve Normality or Symmetry. *Biometrika*, *87*(4), 954–959. JSTOR. https://doi.org/10.2307/2673623

Yogish Kudva. (2024, March 27). *Diabetes - diagnosis and treatment*. Mayoclinic.org; Mayo Clinic. https://www.mayoclinic.org/diseases-conditions/diabetes/diagnosis-treatment/drc-20371451